7–Random-Forest

Elizabeth Buhr

2026-10-07

Load Packages

```
library(tidyverse)
library(sf)
library(here)
library(ranger)
library(tidymodels)
library(DALEXtra) #Explainer (variable importance, partial dependence, etc)
library(spatialsample) #Use for spatial cross validation
library(car) #VIF function

conflicted::conflicts_prefer(yardstick::rmse)
conflicted::conflicts_prefer(dplyr::filter)
```

Load Data

```
plots <- st_read(here("data/processed/fuels_analysis.gpkg"), layer = "allplotsDS") %>%
  filter(Incid_Name != "EAST FORK") #seeing very little treated area in this fire now
plots$northness[is.na(plots$northness)] <- 0
plots$eastness[is.na(plots$eastness)] <- 0

plots_100m <- filter(plots, buffer_radius == 100)
plots_250m <- filter(plots, buffer_radius == 250)
plots_500m <- filter(plots, buffer_radius == 500)

preds <- c("northness", "eastness", "elevation", "TPI", "slope", "Tmax_4k", "PR_4k", "VS_4k", "ERC_4k",

plots_500m %>%
  st_drop_geometry() %>%
  group_by(Incid_Name) %>%
  summarize(mean(prop_extent_treated))
```

Correlation of Predictor Variables

VIF and linear correlation

```
model <- lm(prop_high_sev ~ northness + eastness + elevation + TPI + slope + Tmax_4k + PR_4k + VS_4k + E

vif(model)
```

```
##             northness              eastness             elevation
##              1.124876              1.110671              1.451210
##                   TPI                 slope              Tmax_4k
##              1.203075              1.259523              2.379293
##                PR_4k                 VS_4k                ERC_4k
##              1.929726              1.248098              2.426297
##                CWD_4k               PDSI_4k   prop_extent_treated
##              2.922895              1.514960              1.158642
##          prop_conifer prop_conifer_hardwood         prop_hardwood
##             13.922646              7.254685              4.516502
##        prop_shrubland        prop_grassland                   TCC
##              3.461425              1.515316              1.657441
```

```
#removing individual prop_conifer_hardwoord
model2 <- lm(prop_high_sev ~ northness + eastness + elevation + TPI + slope + Tmax_4k + PR_4k + VS_4k +
  #note, I checked vif for each scale of plot and each response variable. slightly higher vifs at large
vif(model2)
```

```
##           northness            eastness           elevation                 TPI
##            1.124662            1.105543            1.413000            1.054174
##               slope             Tmax_4k               PR_4k               VS_4k
##            1.206123            2.348818            1.926127            1.247383
##              ERC_4k              CWD_4k              PDSI_4k prop_extent_treated
##            2.413529            2.857685            1.488371            1.153148
##        prop_conifer       prop_hardwood      prop_shrubland      prop_grassland
##            2.668349            1.800697            1.694526            1.174470
##                 TCC
##            1.610481
```

Proportion of High-Severity

100m

```
set.seed(123)
data_split_hi100 <- initial_split(plots_100m, strata = "prop_high_sev", prop = 0.80)
hi100_train <- training(data_split_hi100)
hi100_test  <- testing(data_split_hi100)
```

Create a recipe

```
hi100_rec <- recipe(prop_high_sev ~ northness + eastness + elevation + TPI + slope + Tmax_4k + PR_4k + V

hi100_prep <- prep(hi100_rec)
```

Set up a model specification to tune

```
tune_spec <- rand_forest(mtry = tune(),
                         trees = 2000,
                         min_n = tune()) %>%
              set_mode("regression") %>%
              set_engine("ranger", importance = "permutation")
```

Set up a workflow

```
tune_wf <- workflow() %>%
  add_recipe(hi100_rec) %>%
  add_model(tune_spec)
```

Tune hyperparameters based on RMSE

```
set.seed(234)

hi100_folds <- spatial_clustering_cv(hi100_train)

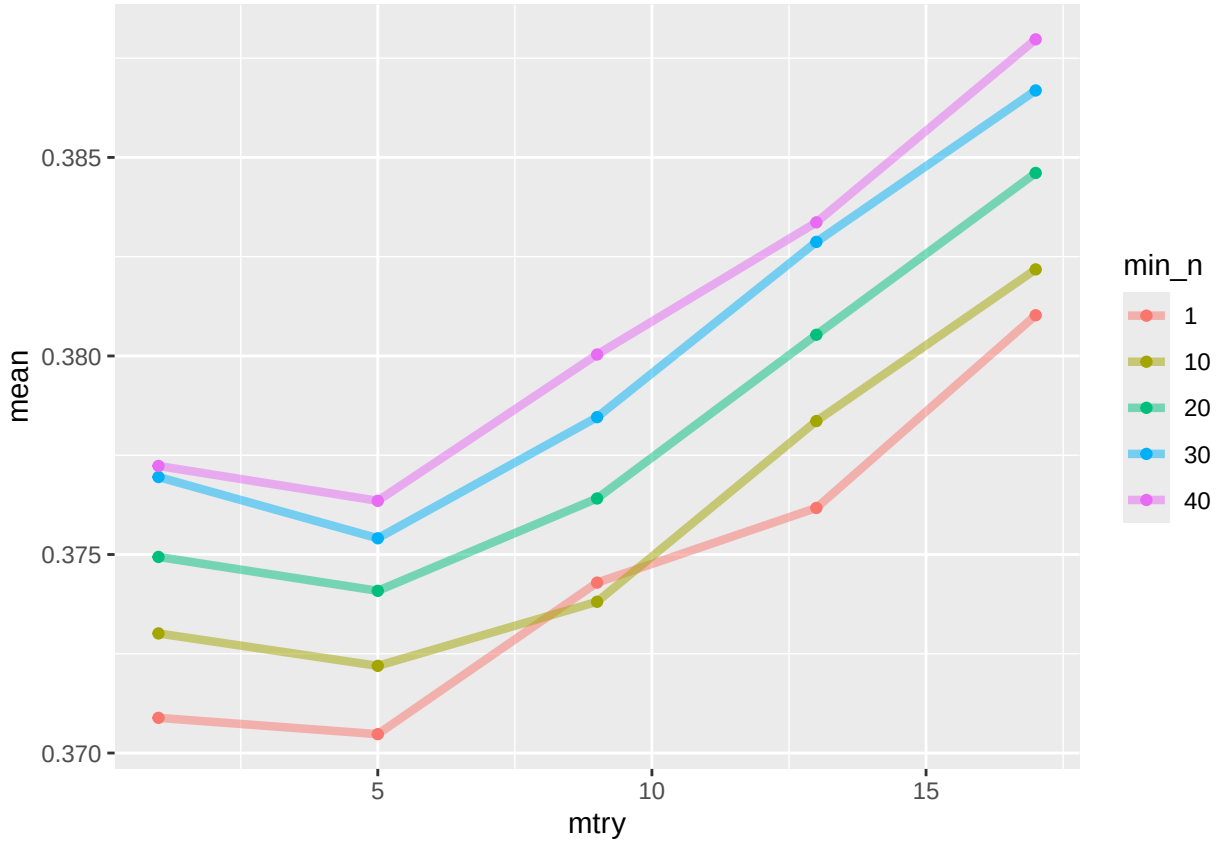
# hi100_folds %>%
#   mutate(held_out_fire = map(splits, ~ assessment(.x) %>% distinct(Incid_Name))) %>%
#   unnest(held_out_fire) %>%
#   select(id, Incid_Name)

doParallel::registerDoParallel()
#show_notes(.Last.tune.result)

rf_grid <- grid_regular(
  mtry(range = c(1, 17)),
  min_n(range = c(1,40)),
  levels = 5
)

set.seed(456)
regular_res <- tune_grid(
  tune_wf,
  resamples = hi100_folds,
  grid = rf_grid
)

regular_res %>%
  collect_metrics() %>%
  filter(.metric == "rmse") %>%
  mutate(min_n = factor(min_n)) %>%
  ggplot(aes(mtry, mean, color = min_n)) +
  geom_line(alpha = 0.5, size = 1.5) +
  geom_point()
```



Select Best Fit

```
best_rmse <- select_best(regular_res, metric = "rmse")
final_workflow <- tune_wf %>%
  finalize_workflow(best_rmse)
```

Evaluate metrics across spatial crossvalidation folds

```
rf_res <-
    final_workflow %>%
    fit_resamples(
      resamples = hi100_folds,
      metrics = metric_set(rmse, rsq),
      control = control_resamples(save_pred = TRUE)
      )

rf_res %>%
  collect_metrics(summarize = TRUE) %>%
  mutate(Dataset="training") %>%
  mutate(Model="RF")
```

```
## # A tibble: 2 x 8
##   .metric .estimator  mean     n std_err .config         Dataset  Model
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>           <chr>    <chr>
## 1 rmse    standard   0.369    10  0.0183 pre0_mod0_post0 training RF
## 2 rsq     standard   0.256    10  0.0592 pre0_mod0_post0 training RF
```

```
rf_res %>%
  collect_metrics(summarize = FALSE) %>%
  dplyr::filter(.metric == "rmse")
```

```
## # A tibble: 10 x 5
##    id     .metric .estimator .estimate .config
##    <chr>  <chr>   <chr>          <dbl> <chr>
##  1 Fold01 rmse    standard       0.301 pre0_mod0_post0
##  2 Fold02 rmse    standard       0.349 pre0_mod0_post0
##  3 Fold03 rmse    standard       0.337 pre0_mod0_post0
##  4 Fold04 rmse    standard       0.388 pre0_mod0_post0
##  5 Fold05 rmse    standard       0.422 pre0_mod0_post0
##  6 Fold06 rmse    standard       0.388 pre0_mod0_post0
##  7 Fold07 rmse    standard       0.324 pre0_mod0_post0
##  8 Fold08 rmse    standard       0.499 pre0_mod0_post0
##  9 Fold09 rmse    standard       0.354 pre0_mod0_post0
## 10 Fold10 rmse    standard       0.328 pre0_mod0_post0
```

```
rf_res %>%
  collect_metrics(summarize = FALSE) %>%
  dplyr::filter(.metric == "rsq")
```

```
## # A tibble: 10 x 5
##    id     .metric .estimator .estimate .config
##    <chr>  <chr>   <chr>          <dbl> <chr>
##  1 Fold01 rsq     standard     0.416   pre0_mod0_post0
##  2 Fold02 rsq     standard     0.379   pre0_mod0_post0
##  3 Fold03 rsq     standard     0.529   pre0_mod0_post0
##  4 Fold04 rsq     standard     0.125   pre0_mod0_post0
##  5 Fold05 rsq     standard     0.00229 pre0_mod0_post0
##  6 Fold06 rsq     standard     0.152   pre0_mod0_post0
##  7 Fold07 rsq     standard     0.300   pre0_mod0_post0
##  8 Fold08 rsq     standard     0.00806 pre0_mod0_post0
##  9 Fold09 rsq     standard     0.192   pre0_mod0_post0
## 10 Fold10 rsq     standard     0.456   pre0_mod0_post0
```

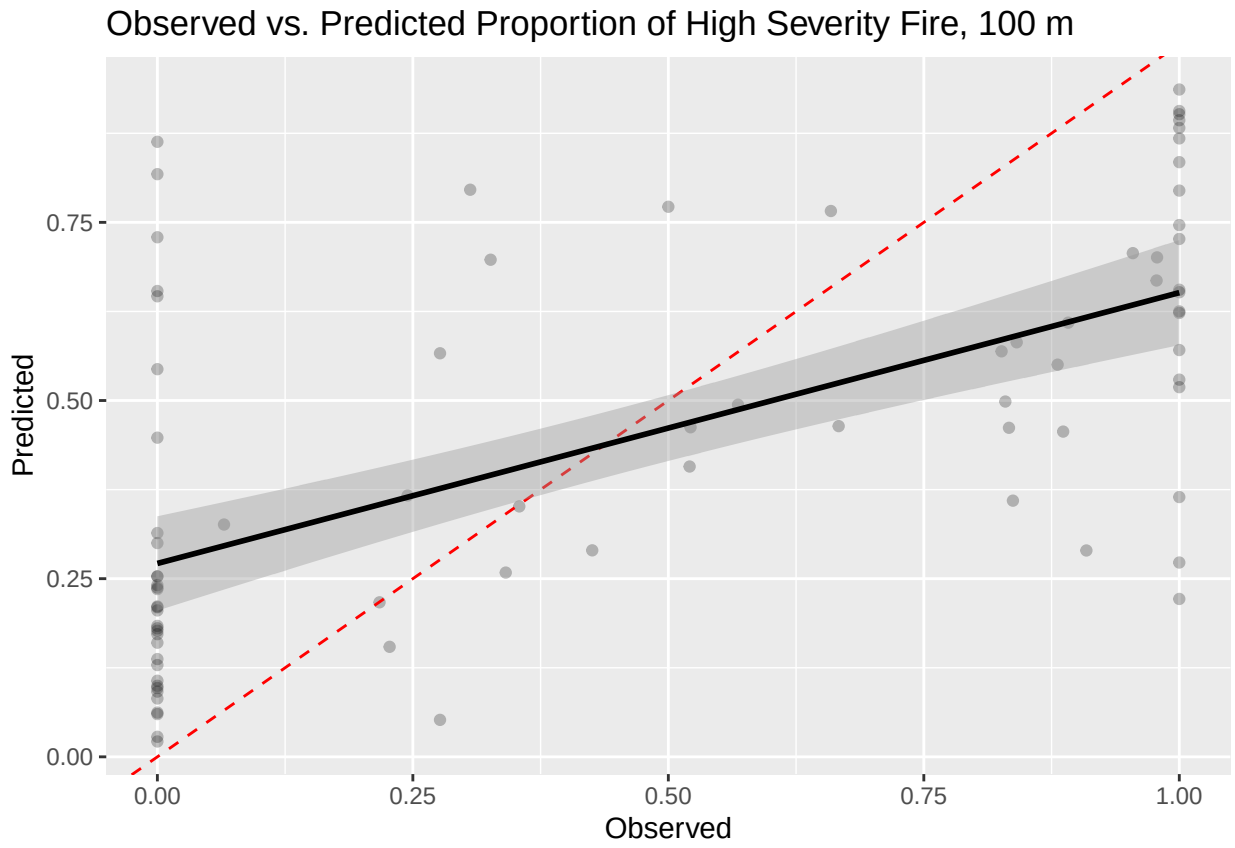
Fit Final Model

```
final_fit <- final_workflow %>%
  last_fit(data_split_hi100)

collect_metrics(final_fit)      # test-set RMSE, R², etc.
```

```
## # A tibble: 2 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>          <dbl> <chr>
## 1 rmse    standard       0.340 pre0_mod0_post0
## 2 rsq     standard       0.392 pre0_mod0_post0
```

```r
hi100_predictions <- collect_predictions(final_fit)  # observed vs. predicted on the held-out test set
ggplot(hi100_predictions, aes(prop_high_sev, .pred)) +
  geom_point(alpha = 0.25)+
  geom_abline(linetype = "dashed", color = "red") +
  geom_smooth(method = "lm", color = "black") +
  labs(
    x = "Observed",
    y = "Predicted",
    color = NULL,
    title = "Observed vs. Predicted Proportion of High Severity Fire, 100 m"
  )
```




```r
final_fitted <- final_fit$.workflow[[1]]
final_fitted
```

```
## == Workflow [trained] ==========================================================
## Preprocessor: Recipe
## Model: rand_forest()
##
## -- Preprocessor ----------------------------------------------------------------
## 0 Recipe Steps
##
## -- Model -----------------------------------------------------------------------
## Ranger result
##
```

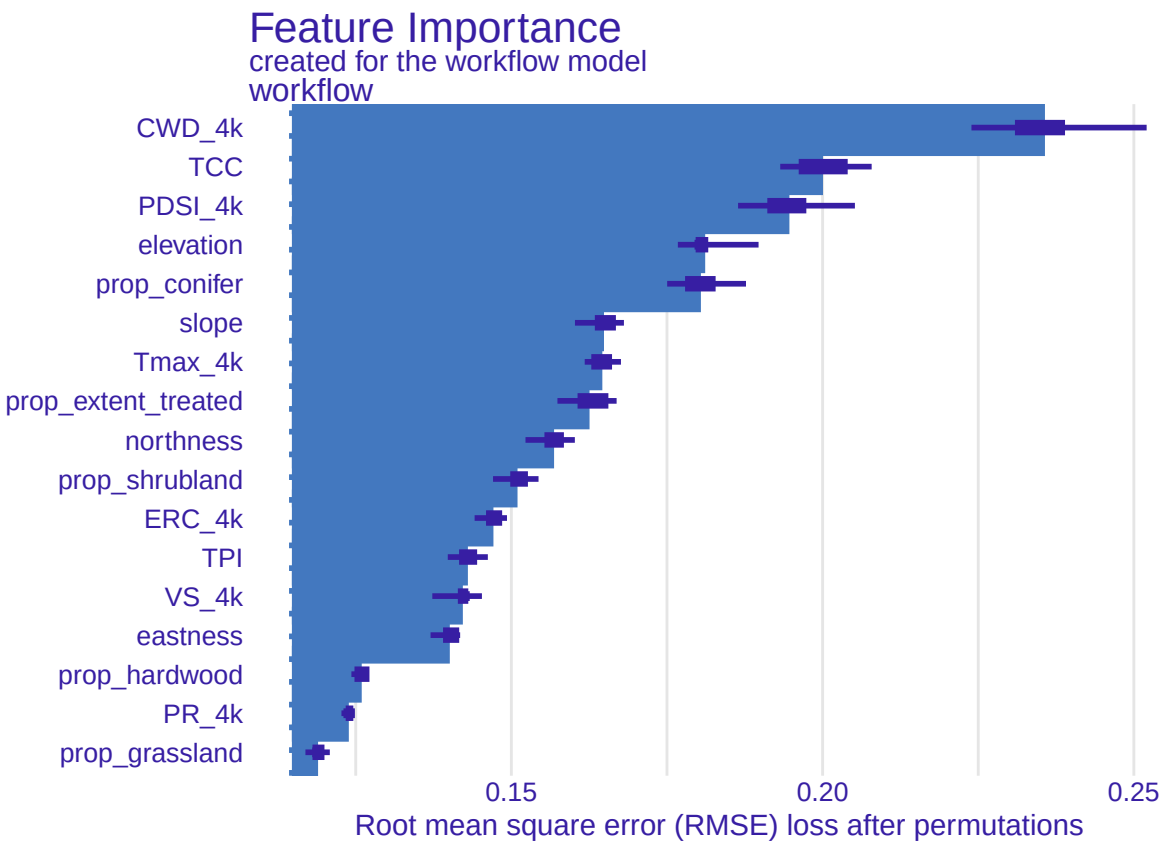
```
## Call:
##  ranger::ranger(x = maybe_data_frame(x), y = y, mtry = min_cols(~5L,      x), num.trees = ~2000, min
##
## Type:                             Regression
## Number of trees:                  2000
## Sample size:                      324
## Number of independent variables:  17
## Mtry:                             5
## Target node size:                 1
## Variable importance mode:         permutation
## Splitrule:                        variance
## OOB prediction error (MSE):       0.09741077
## R squared (OOB):                  0.4818847
```

Explainer

```
hi100_explainer <- explain_tidymodels(
  final_fitted,
  data = hi100_train %>% st_drop_geometry() %>% dplyr::select(all_of(preds), Incid_Name),
  y = hi100_train$prop_high_sev,
  verbose = FALSE
)
```

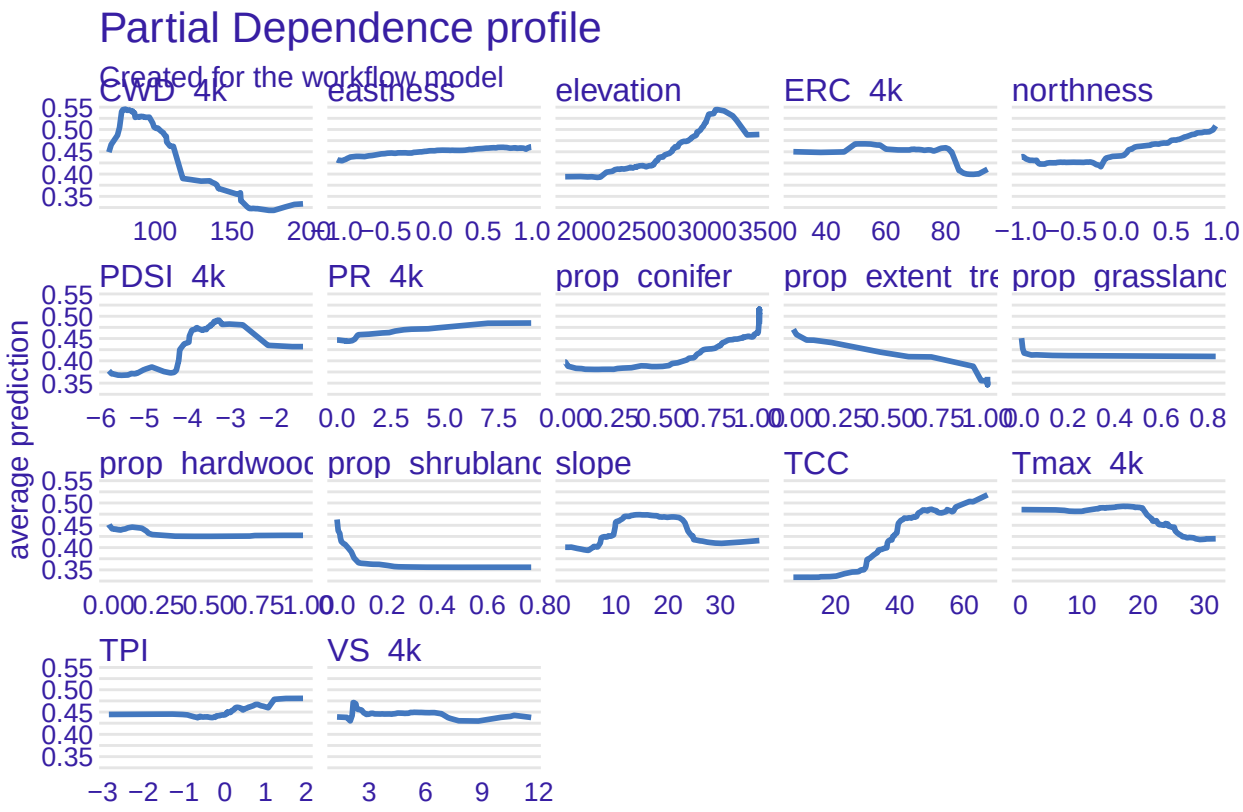
Variable Importance Plot

```
# Variable Importance Plots
vip_hi100 <- variable_importance(hi100_explainer,
                                 variables = preds)
plot(vip_hi100)
```



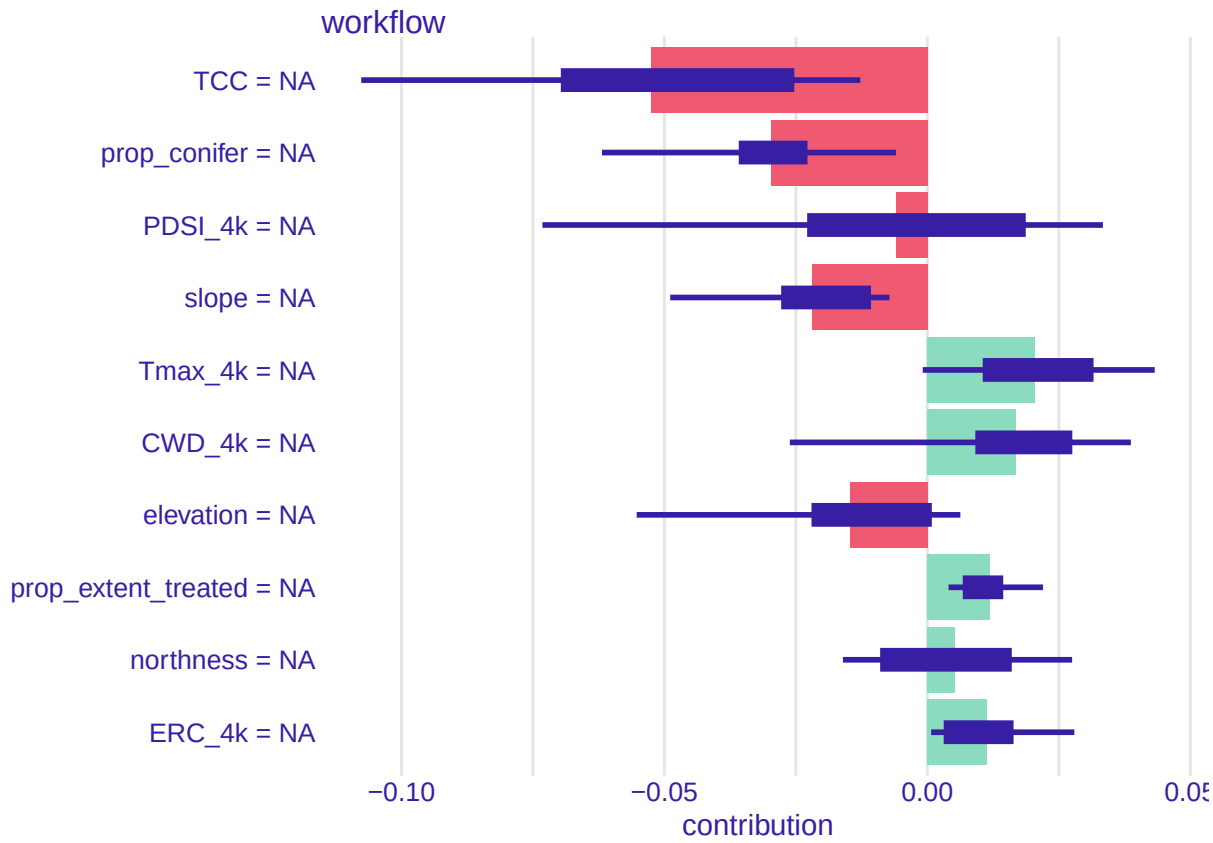
PDP

```
pdp_hi100 <- model_profile(
  hi100_explainer,
  variables = preds,
  N = NULL
)
plot(pdp_hi100)
```



Shaply Plot - just for fun

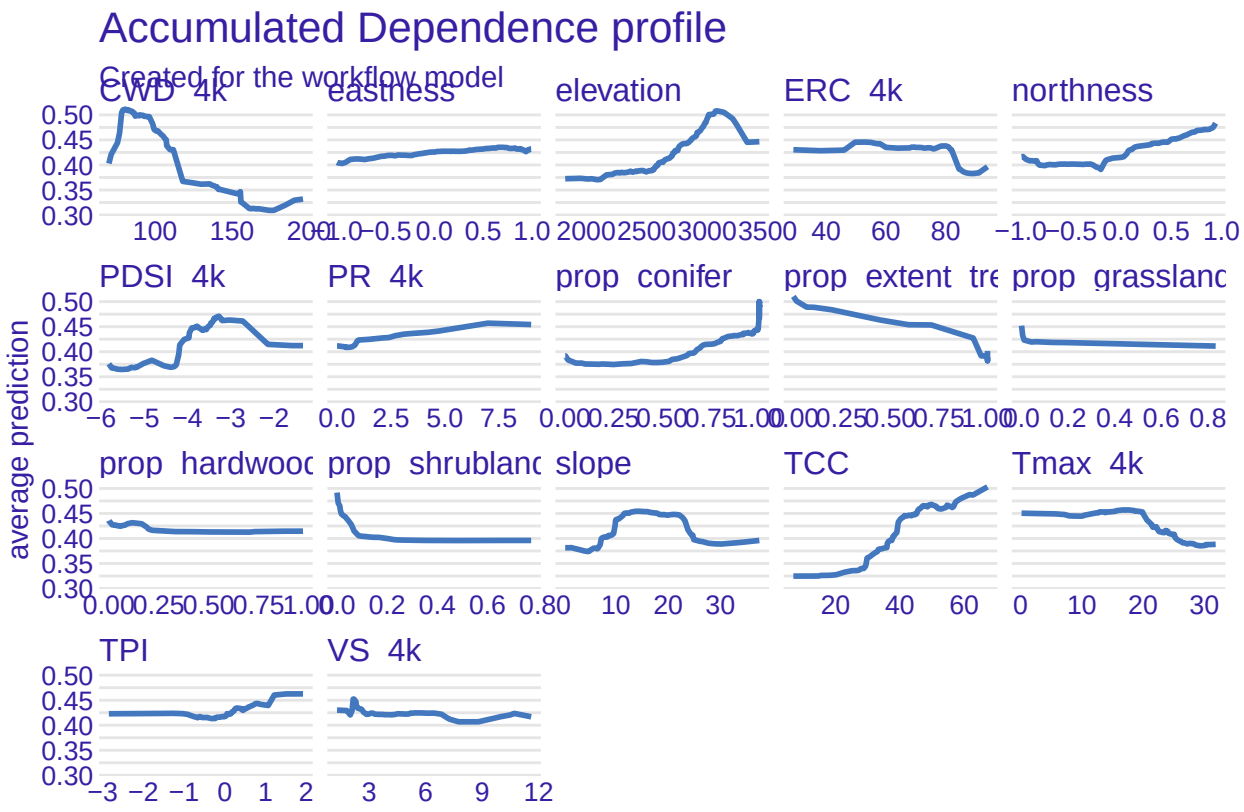
```
shap_plots <- predict_parts(hi100_explainer, hi100_test[96, ], type = 'shap')
plot(shap_plots)
```



ALE plot

```
ale_hi100 <- model_profile(
  explainer = hi100_explainer,
  variables = preds,
  type = 'accumulated'
)

plot(ale_hi100)
```



250m

```
set.seed(123)
data_split_hi250 <- initial_split(plots_250m, strata = "prop_high_sev", prop = 0.8)
hi250_train <- training(data_split_hi250)
hi250_test  <- testing(data_split_hi250)
```

Create a recipe

```
hi250_rec <- recipe(prop_high_sev ~ northness + eastness + elevation + TPI + slope + Tmax_4k + PR_4k + 

hi250_prep <- prep(hi250_rec)
```

Set up a model specification to tune

```
tune_spec <- rand_forest(mtry = tune(),
                         trees = 2000,
                         min_n = tune()) %>%
             set_mode("regression") %>%
             set_engine("ranger", importance = "permutation")
```

Set up a workflow

```r
tune_wf <- workflow() %>%
  add_recipe(hi250_rec) %>%
  add_model(tune_spec)
```

Tune hyperparameters based on RMSE

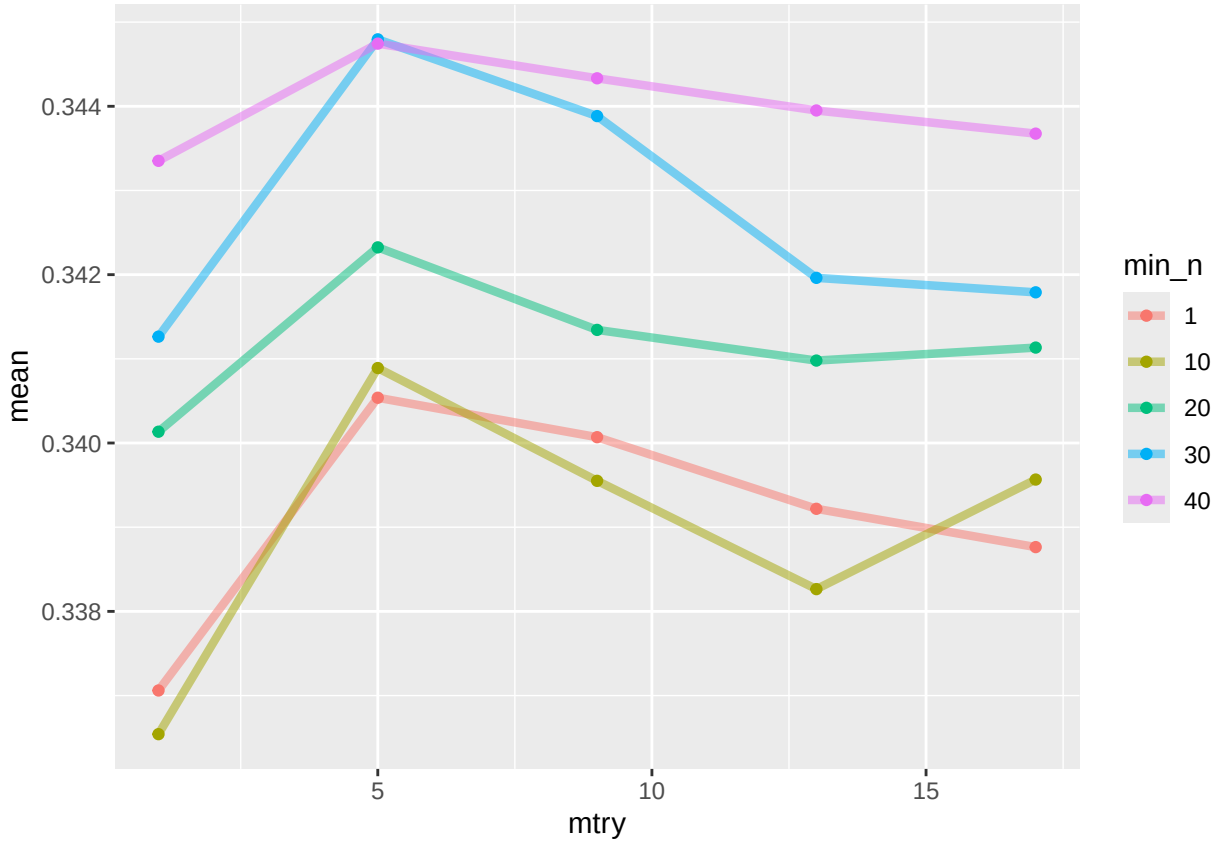
```r
set.seed(234)
hi250_folds <- spatial_clustering_cv(hi250_train)

doParallel::registerDoParallel()
#show_notes(.Last.tune.result)

rf_grid <- grid_regular(
  mtry(range = c(1, 17)),
  min_n(range = c(1,40)),
  levels = 5
)

set.seed(456)
regular_res <- tune_grid(
  tune_wf,
  resamples = hi250_folds,
  grid = rf_grid
)

regular_res %>%
  collect_metrics() %>%
  filter(.metric == "rmse") %>%
  mutate(min_n = factor(min_n)) %>%
  ggplot(aes(mtry, mean, color = min_n)) +
  geom_line(alpha = 0.5, size = 1.5) +
  geom_point()
```



Select Best Fit

```
best_rmse <- select_best(regular_res, metric = "rmse")
final_workflow <- tune_wf %>%
  finalize_workflow(best_rmse)
```

Evaluate metrics across spatial crossvalidation folds

```
rf_res <-
    final_workflow %>%
    fit_resamples(
      resamples = hi250_folds,
      metrics = metric_set(rmse, rsq),
      control = control_resamples(save_pred = TRUE)
      )

rf_res %>%
  collect_metrics(summarize = TRUE) %>%
  mutate(Dataset="training") %>%
  mutate(Model="RF")
```

```
## # A tibble: 2 x 8
##   .metric .estimator  mean     n std_err .config         Dataset  Model
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>           <chr>    <chr>
## 1 rmse    standard   0.338    10  0.0217 pre0_mod0_post0 training RF
## 2 rsq     standard   0.191     9  0.0460 pre0_mod0_post0 training RF
```

```
rf_res %>%
  collect_metrics(summarize = FALSE) %>%
  filter(.metric == "rmse")
```

```
## # A tibble: 10 x 5
##    id     .metric .estimator .estimate .config
##    <chr>  <chr>   <chr>          <dbl> <chr>
##  1 Fold01 rmse    standard       0.389 pre0_mod0_post0
##  2 Fold02 rmse    standard       0.308 pre0_mod0_post0
##  3 Fold03 rmse    standard       0.286 pre0_mod0_post0
##  4 Fold04 rmse    standard       0.324 pre0_mod0_post0
##  5 Fold05 rmse    standard       0.369 pre0_mod0_post0
##  6 Fold06 rmse    standard       0.303 pre0_mod0_post0
##  7 Fold07 rmse    standard       0.312 pre0_mod0_post0
##  8 Fold08 rmse    standard       0.308 pre0_mod0_post0
##  9 Fold09 rmse    standard       0.506 pre0_mod0_post0
## 10 Fold10 rmse    standard       0.276 pre0_mod0_post0
```

```
rf_res %>%
  collect_metrics(summarize = FALSE) %>%
  dplyr::filter(.metric == "rsq")
```

```
## # A tibble: 10 x 5
##    id     .metric .estimator .estimate .config
##    <chr>  <chr>   <chr>          <dbl> <chr>
##  1 Fold01 rsq     standard      0.123  pre0_mod0_post0
##  2 Fold02 rsq     standard      0.0182 pre0_mod0_post0
##  3 Fold03 rsq     standard      0.266  pre0_mod0_post0
##  4 Fold04 rsq     standard      0.133  pre0_mod0_post0
##  5 Fold05 rsq     standard      0.108  pre0_mod0_post0
##  6 Fold06 rsq     standard      0.328  pre0_mod0_post0
##  7 Fold07 rsq     standard      0.356  pre0_mod0_post0
##  8 Fold08 rsq     standard     NA      pre0_mod0_post0
##  9 Fold09 rsq     standard      0.0281 pre0_mod0_post0
## 10 Fold10 rsq     standard      0.363  pre0_mod0_post0
```

Fit Final Model

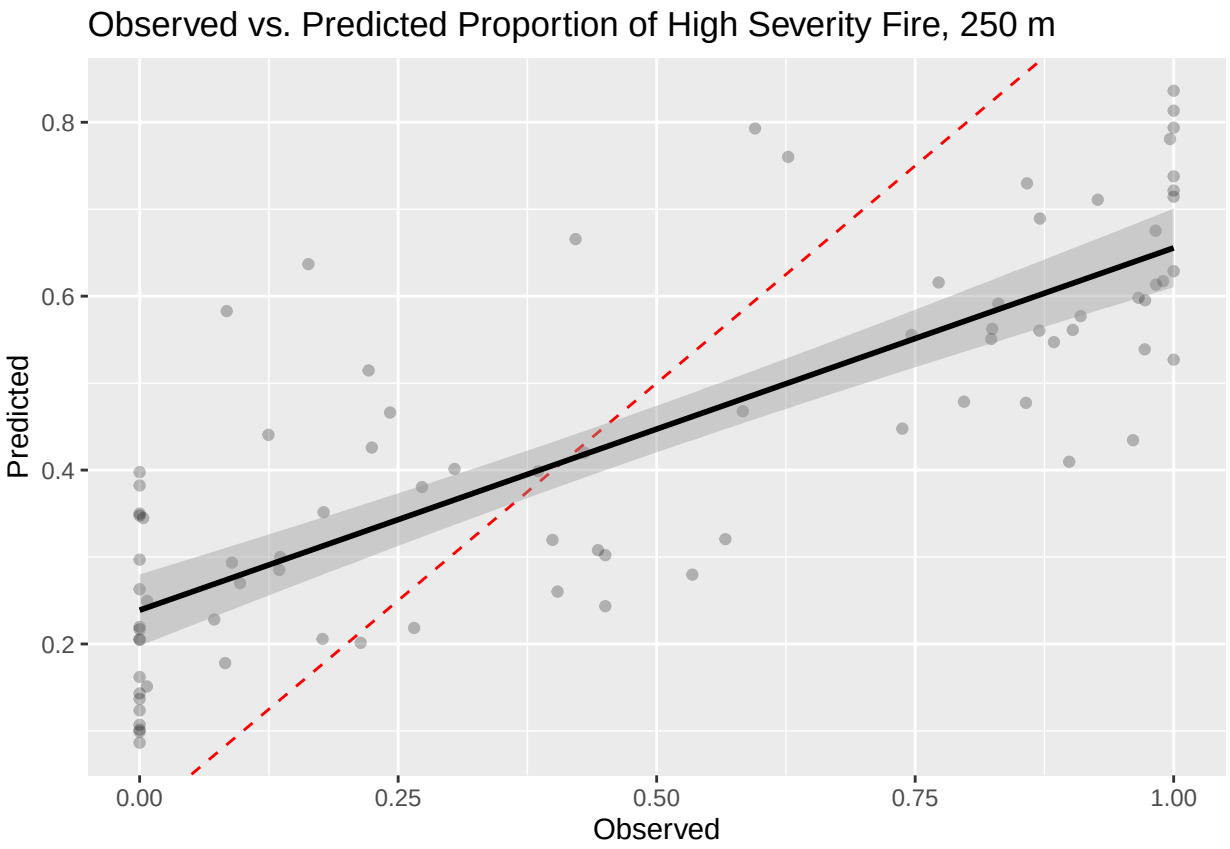
```
final_fit <- final_workflow %>%
  last_fit(data_split_hi250)

collect_metrics(final_fit)      # test-set RMSE, R², etc.
```

```
## # A tibble: 2 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>          <dbl> <chr>
## 1 rmse    standard       0.261 pre0_mod0_post0
## 2 rsq     standard       0.646 pre0_mod0_post0
```

```
hi250_predictions <- collect_predictions(final_fit)  # observed vs. predicted on the held-out test set
ggplot(hi250_predictions, aes(prop_high_sev, .pred)) +
```

```
  geom_point(alpha = 0.25)+
  geom_abline(linetype = "dashed", color = "red") +
  geom_smooth(method = "lm", color = "black") +
  labs(
    x = "Observed",
    y = "Predicted",
    color = NULL,
    title = "Observed vs. Predicted Proportion of High Severity Fire, 250 m"
  )
```




```
final_fitted <- final_fit$.workflow[[1]]
final_fitted
```

```
## == Workflow [trained] ==========================================================
## Preprocessor: Recipe
## Model: rand_forest()
##
## -- Preprocessor ----------------------------------------------------------------
## 0 Recipe Steps
##
## -- Model -----------------------------------------------------------------------
## Ranger result
##
## Call:
##  ranger::ranger(x = maybe_data_frame(x), y = y, mtry = min_cols(~1L,      x), num.trees = ~2000, min
```

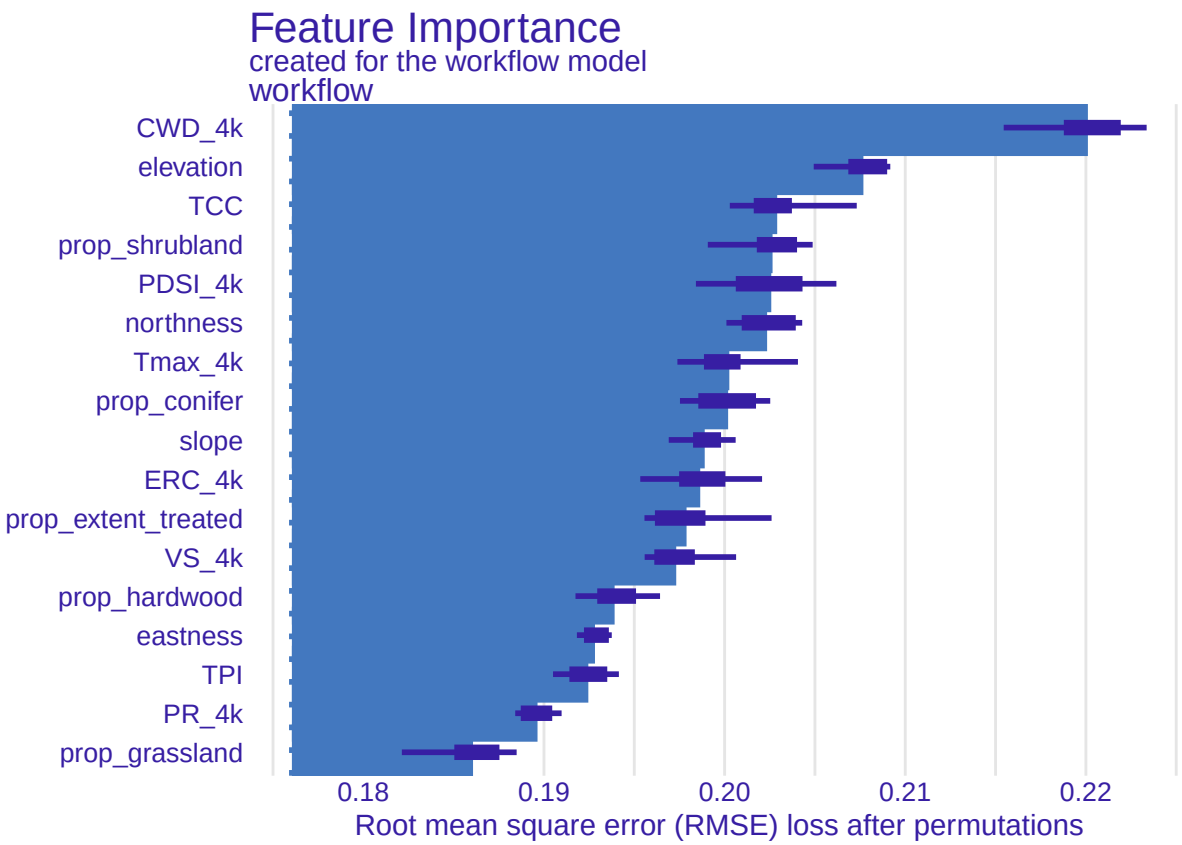
```
##
## Type:                             Regression
## Number of trees:                  2000
## Sample size:                      322
## Number of independent variables:  17
## Mtry:                             1
## Target node size:                 10
## Variable importance mode:         permutation
## Splitrule:                        variance
## OOB prediction error (MSE):       0.07652984
## R squared (OOB):                  0.4990852
```

Explainer

```
hi250_explainer <- explain_tidymodels(
  final_fitted,
  data = hi250_train %>% st_drop_geometry() %>% dplyr::select(all_of(preds), Incid_Name),
  y = hi250_train$prop_high_sev,
  verbose = FALSE
)
```

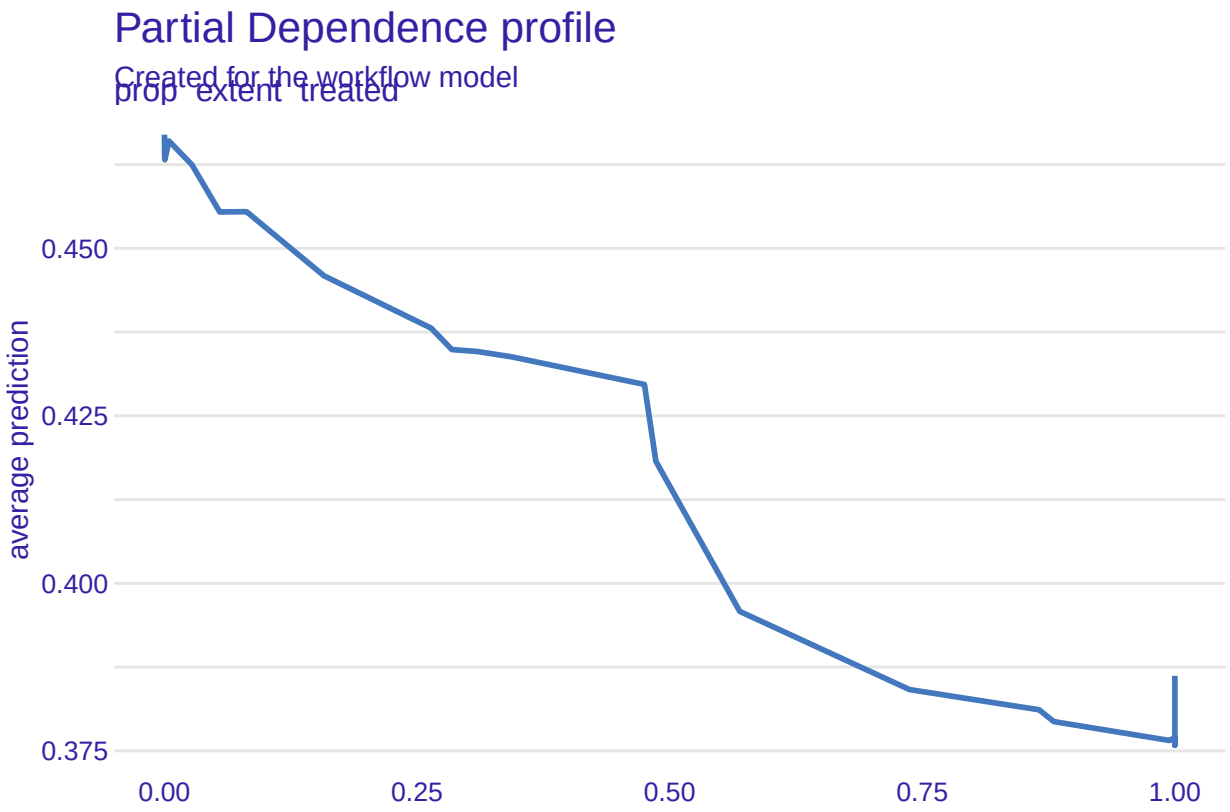
Variable Importance Plot

```
# Variable Importance Plots
vip_hi250 <- variable_importance(hi250_explainer,
                                 variables = preds)
plot(vip_hi250)
```



PDP

```
pdp_hi250 <- model_profile(
  hi250_explainer,
  variables = "prop_extent_treated",
  N = NULL
)
plot(pdp_hi250)
```



500m

```
set.seed(123)
data_split_hi500 <- initial_split(plots_500m, strata = "prop_high_sev", prop = 0.8)
hi500_train <- training(data_split_hi500)
hi500_test  <- testing(data_split_hi500)
```

Create a recipe

```
hi500_rec <- recipe(prop_high_sev ~ northness + eastness + elevation + TPI + slope + Tmax_4k + PR_4k + 

hi500_prep <- prep(hi500_rec)
```

Set up a model specification to tune

```
tune_spec <- rand_forest(mtry = tune(),
                         trees = 2000,
                         min_n = tune()) %>%
             set_mode("regression") %>%
             set_engine("ranger", importance = "permutation")
```

Set up a workflow

```
tune_wf <- workflow() %>%
  add_recipe(hi500_rec) %>%
  add_model(tune_spec)
```

Tune hyperparameters based on RMSE

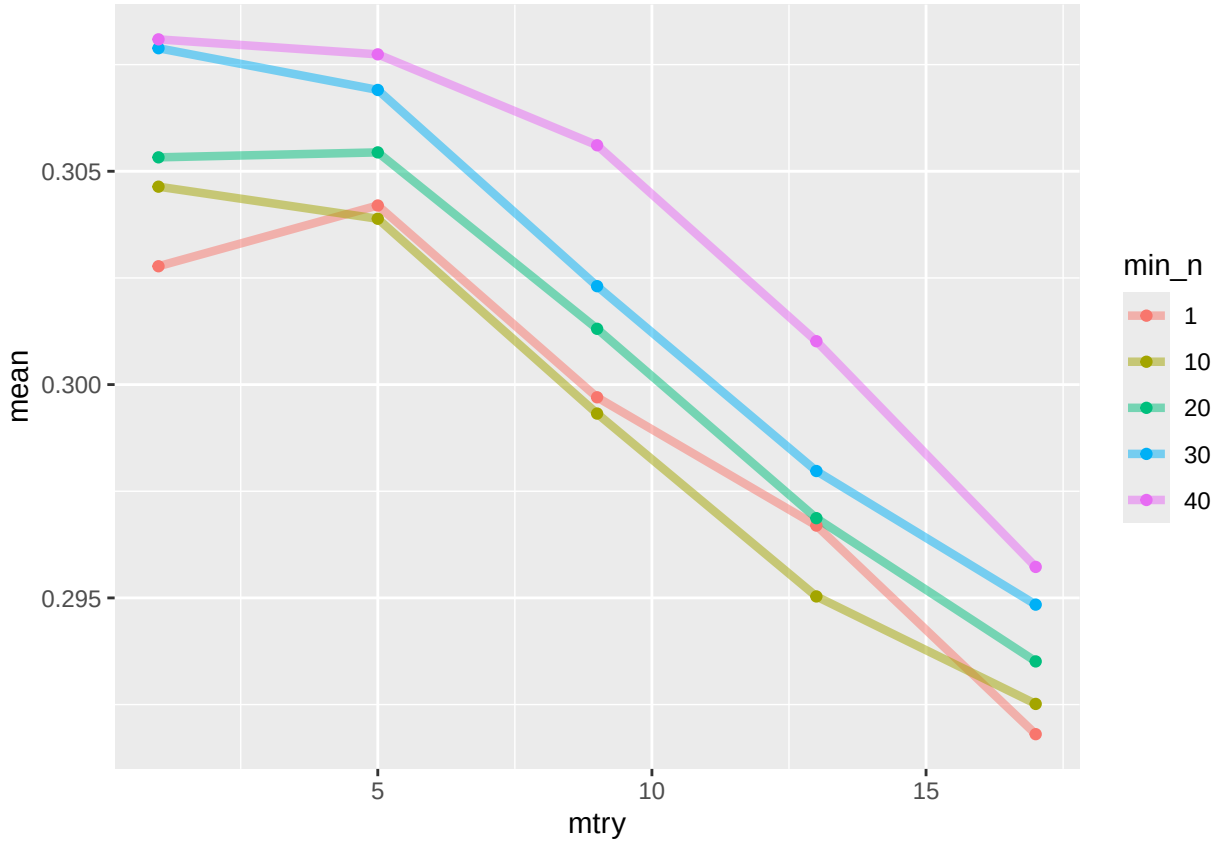
```
set.seed(234)
hi500_folds <- spatial_clustering_cv(hi500_train)

doParallel::registerDoParallel()
#show_notes(.Last.tune.result)

rf_grid <- grid_regular(
  mtry(range = c(1, 17)),
  min_n(range = c(1,40)),
  levels = 5
)

set.seed(456)
regular_res <- tune_grid(
  tune_wf,
  resamples = hi500_folds,
  grid = rf_grid
)

regular_res %>%
  collect_metrics() %>%
  filter(.metric == "rmse") %>%
  mutate(min_n = factor(min_n)) %>%
  ggplot(aes(mtry, mean, color = min_n)) +
  geom_line(alpha = 0.5, size = 1.5) +
  geom_point()
```



Select Best Fit

```
best_rmse <- select_best(regular_res, metric = "rmse")
final_workflow <- tune_wf %>%
  finalize_workflow(best_rmse)
```

Evaluate metrics across spatial crossvalidation folds

```
rf_res <-
    final_workflow %>%
    fit_resamples(
      resamples = hi500_folds,
      metrics = metric_set(rmse, rsq),
      control = control_resamples(save_pred = TRUE)
      )

rf_res %>%
  collect_metrics(summarize = TRUE) %>%
  mutate(Dataset="training") %>%
  mutate(Model="RF")
```

```
## # A tibble: 2 x 8
##   .metric .estimator  mean     n std_err .config         Dataset  Model
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>           <chr>    <chr>
## 1 rmse    standard   0.292    10  0.0257 pre0_mod0_post0 training RF
## 2 rsq     standard   0.196    10  0.0361 pre0_mod0_post0 training RF
```

```
rf_res %>%
  collect_metrics(summarize = FALSE) %>%
  filter(.metric == "rmse")
```

```
## # A tibble: 10 x 5
##    id     .metric .estimator .estimate .config
##    <chr>  <chr>   <chr>          <dbl> <chr>
##  1 Fold01 rmse    standard       0.251 pre0_mod0_post0
##  2 Fold02 rmse    standard       0.187 pre0_mod0_post0
##  3 Fold03 rmse    standard       0.236 pre0_mod0_post0
##  4 Fold04 rmse    standard       0.284 pre0_mod0_post0
##  5 Fold05 rmse    standard       0.314 pre0_mod0_post0
##  6 Fold06 rmse    standard       0.301 pre0_mod0_post0
##  7 Fold07 rmse    standard       0.269 pre0_mod0_post0
##  8 Fold08 rmse    standard       0.291 pre0_mod0_post0
##  9 Fold09 rmse    standard       0.498 pre0_mod0_post0
## 10 Fold10 rmse    standard       0.290 pre0_mod0_post0
```

```
rf_res %>%
  collect_metrics(summarize = FALSE) %>%
  dplyr::filter(.metric == "rsq")
```

```
## # A tibble: 10 x 5
##    id     .metric .estimator .estimate .config
##    <chr>  <chr>   <chr>          <dbl> <chr>
##  1 Fold01 rsq     standard      0.275  pre0_mod0_post0
##  2 Fold02 rsq     standard      0.125  pre0_mod0_post0
##  3 Fold03 rsq     standard      0.238  pre0_mod0_post0
##  4 Fold04 rsq     standard      0.105  pre0_mod0_post0
##  5 Fold05 rsq     standard      0.0590 pre0_mod0_post0
##  6 Fold06 rsq     standard      0.166  pre0_mod0_post0
##  7 Fold07 rsq     standard      0.396  pre0_mod0_post0
##  8 Fold08 rsq     standard      0.175  pre0_mod0_post0
##  9 Fold09 rsq     standard      0.0746 pre0_mod0_post0
## 10 Fold10 rsq     standard      0.342  pre0_mod0_post0
```

Fit Final Model

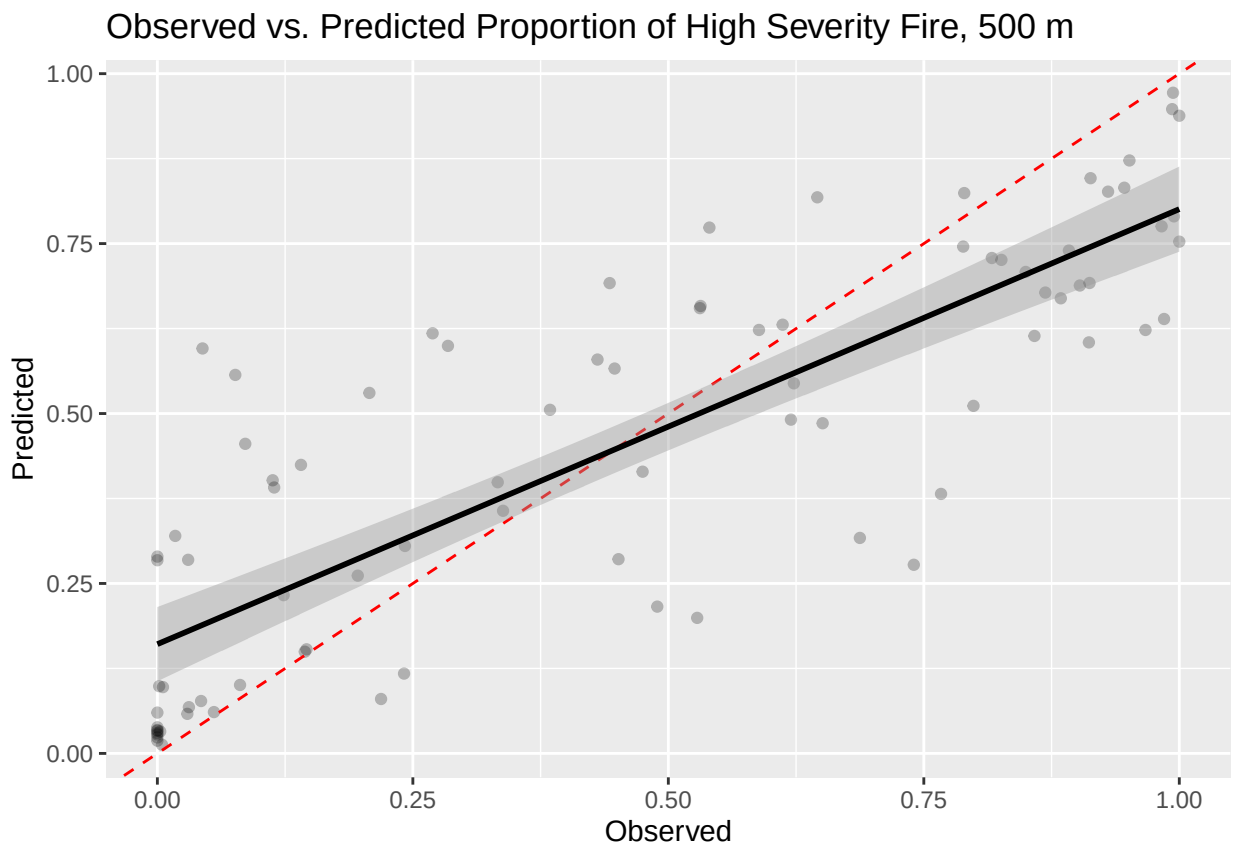
```
final_fit <- final_workflow %>%
  last_fit(data_split_hi500)

collect_metrics(final_fit)      # test-set RMSE, R², etc.
```

```
## # A tibble: 2 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>          <dbl> <chr>
## 1 rmse    standard       0.205 pre0_mod0_post0
## 2 rsq     standard       0.689 pre0_mod0_post0
```

```
hi500_predictions <- collect_predictions(final_fit)  # observed vs. predicted on the held-out test set
ggplot(hi500_predictions, aes(prop_high_sev, .pred)) +
```

```
  geom_point(alpha = 0.25)+
  geom_abline(linetype = "dashed", color = "red") +
  geom_smooth(method = "lm", color = "black") +
  labs(
    x = "Observed",
    y = "Predicted",
    color = NULL,
    title = "Observed vs. Predicted Proportion of High Severity Fire, 500 m"
  )
```




```
final_fitted <- final_fit$.workflow[[1]]
final_fitted
```

```
## == Workflow [trained] ==========================================================
## Preprocessor: Recipe
## Model: rand_forest()
##
## -- Preprocessor ----------------------------------------------------------------
## 0 Recipe Steps
##
## -- Model -----------------------------------------------------------------------
## Ranger result
##
## Call:
##  ranger::ranger(x = maybe_data_frame(x), y = y, mtry = min_cols(~17L,      x), num.trees = ~2000, mi
```

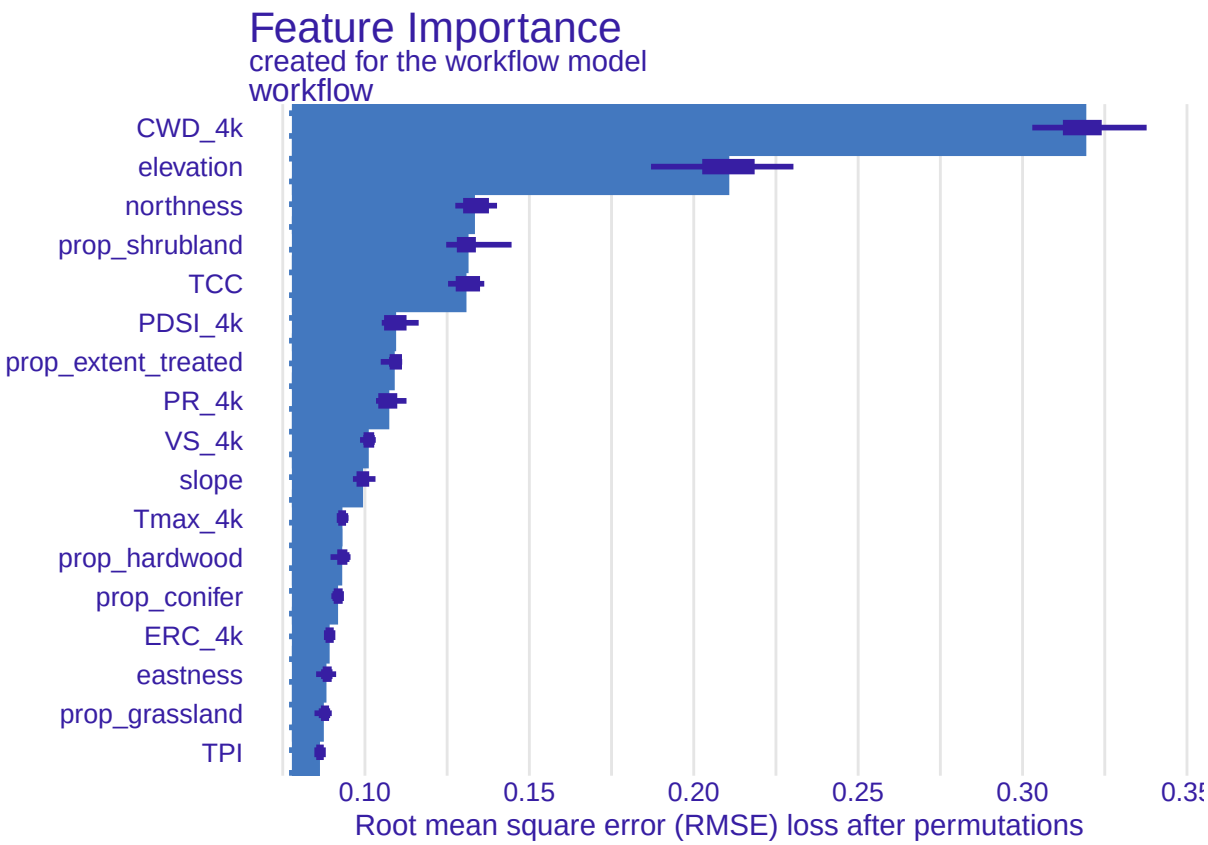
```
##
## Type:                             Regression
## Number of trees:                  2000
## Sample size:                      322
## Number of independent variables:  17
## Mtry:                             17
## Target node size:                 1
## Variable importance mode:         permutation
## Splitrule:                        variance
## OOB prediction error (MSE):       0.04461974
## R squared (OOB):                  0.6581212
```

Explainer

```
hi500_explainer <- explain_tidymodels(
  final_fitted,
  data = hi500_train %>% st_drop_geometry() %>% dplyr::select(all_of(preds), Incid_Name),
  y = hi500_train$prop_high_sev,
  verbose = FALSE
)
```

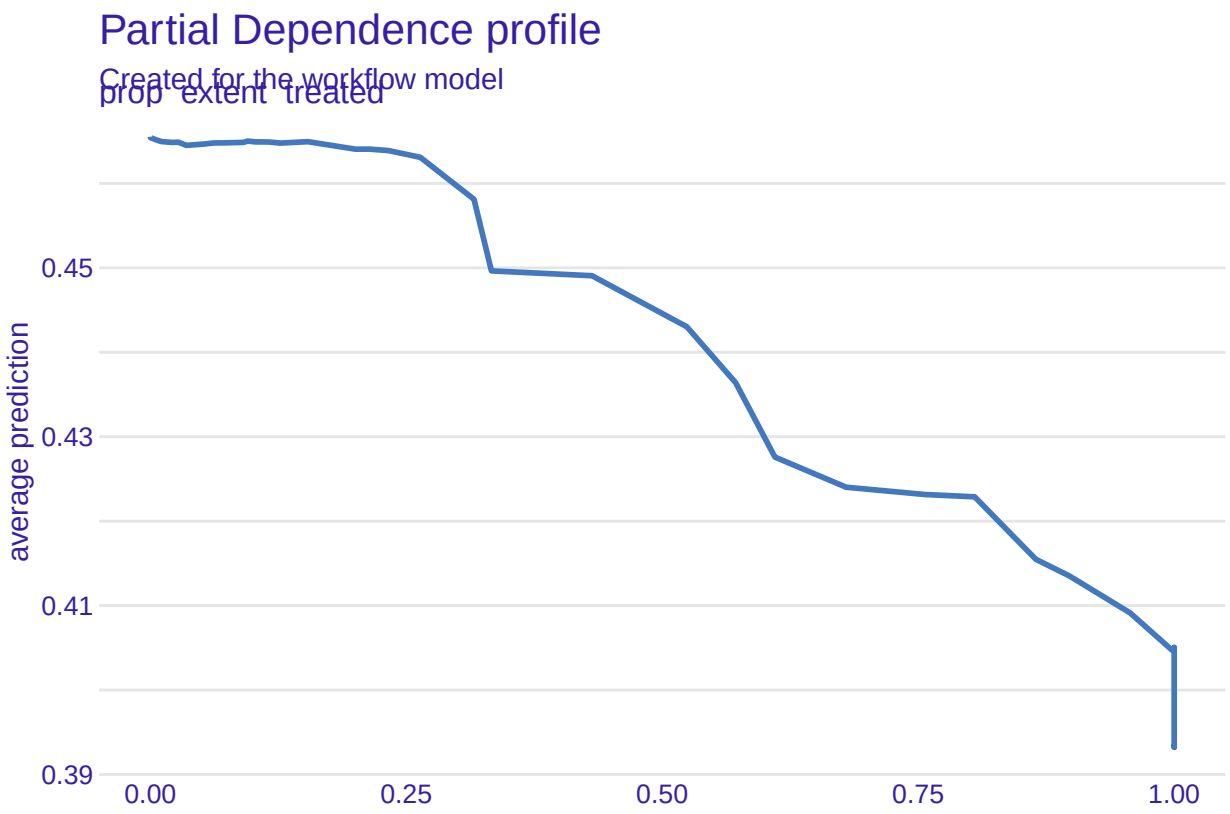
Variable Importance Plot

```
# Variable Importance Plots
vip_hi500 <- variable_importance(hi500_explainer,
                                 variables = preds)
plot(vip_hi500)
```



PDP

```
pdp_hi500 <- model_profile(
  hi500_explainer,
  variables = "prop_extent_treated",
  N = NULL
)
plot(pdp_hi500)
```



Proportion of Moderate to High-severity

100m

```
set.seed(123)
data_split_mohi100 <- initial_split(plots_100m, strata = "prop_mod_high_sev", prop = 0.8)
mohi100_train <- training(data_split_mohi100)
mohi100_test  <- testing(data_split_mohi100)
```

Create a recipe

```
mohi100_rec <- recipe(prop_mod_high_sev ~ northness + eastness + elevation + TPI + slope + Tmax_4k + PR

mohi100_prep <- prep(mohi100_rec)
```

Set up a model specification to tune

```
tune_spec <- rand_forest(mtry = tune(),
                         trees = 2000,
                         min_n = tune()) %>%
              set_mode("regression") %>%
              set_engine("ranger", importance = "permutation")
```

Set up a workflow

```
tune_wf <- workflow() %>%
  add_recipe(mohi100_rec) %>%
  add_model(tune_spec)
```

Tune hyperparameters based on RMSE

```
set.seed(234)
mohi100_folds <- spatial_clustering_cv(mohi100_train)

mohi100_folds %>%
  mutate(held_out_fire = map(splits, ~ assessment(.x) %>% distinct(Incid_Name))) %>%
  unnest(held_out_fire) %>%
  select(id, Incid_Name)
```

```
## # A tibble: 14 x 2
##    id     Incid_Name
##    <chr>  <chr>
##  1 Fold01 CAMERON PEAK
##  2 Fold02 DOE CANYON
##  3 Fold02 PLATEAU
##  4 Fold03 CAMERON PEAK
##  5 Fold04 CAMERON PEAK
##  6 Fold05 DUCKETT
##  7 Fold06 CALWOOD
##  8 Fold07 CAMERON PEAK
##  9 Fold08 LITTLE SAND
## 10 Fold08 BEAVER
## 11 Fold08 LITTLE MESA
## 12 Fold09 EIGHT FOUR TWO
## 13 Fold10 LAKE CHRISTINE
## 14 Fold10 WESTON PASS
```

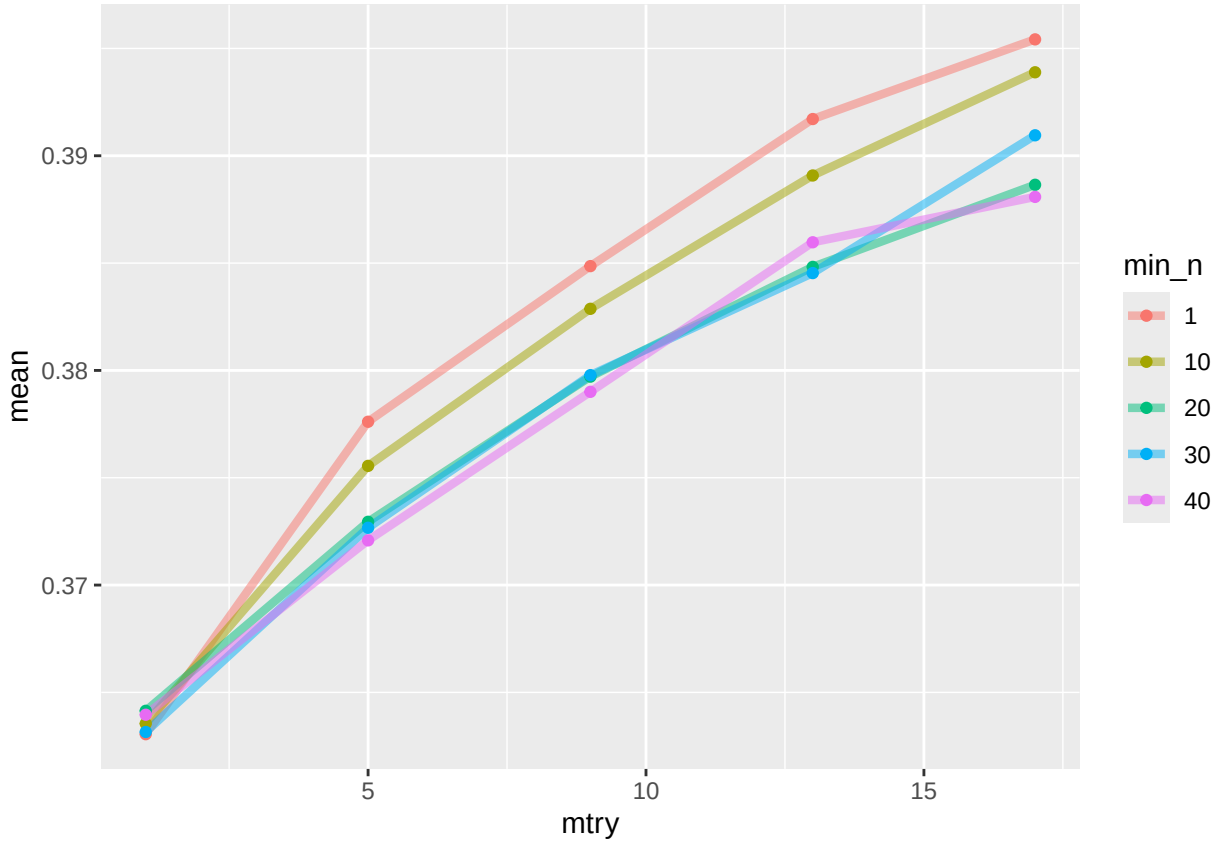
```
doParallel::registerDoParallel()
#show_notes(.Last.tune.result)

rf_grid <- grid_regular(
  mtry(range = c(1, 17)),
  min_n(range = c(1,40)),
  levels = 5
)

set.seed(456)
regular_res <- tune_grid(
  tune_wf,
  resamples = mohi100_folds,
  grid = rf_grid
)

regular_res %>%
  collect_metrics() %>%
  filter(.metric == "rmse") %>%
```

```
  mutate(min_n = factor(min_n)) %>%
  ggplot(aes(mtry, mean, color = min_n)) +
  geom_line(alpha = 0.5, size = 1.5) +
  geom_point()
```



Select Best Fit

```
best_rmse <- select_best(regular_res, metric = "rmse")
final_workflow <- tune_wf %>%
  finalize_workflow(best_rmse)
```

Evaluate metrics across spatial crossvalidation folds

```
rf_res <-
    final_workflow %>%
    fit_resamples(
      resamples = mohi100_folds,
      metrics = metric_set(rmse, rsq),
      control = control_resamples(save_pred = TRUE)
      )

rf_res %>%
  collect_metrics(summarize = TRUE) %>%
  mutate(Dataset="training") %>%
  mutate(Model="RF")
```

```
## # A tibble: 2 x 8
##   .metric .estimator  mean     n std_err .config         Dataset  Model
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>           <chr>    <chr>
## 1 rmse    standard   0.363    10  0.0358 pre0_mod0_post0 training RF
## 2 rsq     standard   0.164    10  0.0534 pre0_mod0_post0 training RF
```

```
rf_res %>%
  collect_metrics(summarize = FALSE) %>%
  filter(.metric == "rmse")
```

```
## # A tibble: 10 x 5
##    id     .metric .estimator .estimate .config
##    <chr>  <chr>   <chr>          <dbl> <chr>
##  1 Fold01 rmse    standard       0.303 pre0_mod0_post0
##  2 Fold02 rmse    standard       0.507 pre0_mod0_post0
##  3 Fold03 rmse    standard       0.288 pre0_mod0_post0
##  4 Fold04 rmse    standard       0.223 pre0_mod0_post0
##  5 Fold05 rmse    standard       0.340 pre0_mod0_post0
##  6 Fold06 rmse    standard       0.215 pre0_mod0_post0
##  7 Fold07 rmse    standard       0.377 pre0_mod0_post0
##  8 Fold08 rmse    standard       0.463 pre0_mod0_post0
##  9 Fold09 rmse    standard       0.544 pre0_mod0_post0
## 10 Fold10 rmse    standard       0.371 pre0_mod0_post0
```

```
rf_res %>%
  collect_metrics(summarize = FALSE) %>%
  dplyr::filter(.metric == "rsq")
```

```
## # A tibble: 10 x 5
##    id     .metric .estimator .estimate .config
##    <chr>  <chr>   <chr>          <dbl> <chr>
##  1 Fold01 rsq     standard    0.0593   pre0_mod0_post0
##  2 Fold02 rsq     standard    0.000913 pre0_mod0_post0
##  3 Fold03 rsq     standard    0.399    pre0_mod0_post0
##  4 Fold04 rsq     standard    0.242    pre0_mod0_post0
##  5 Fold05 rsq     standard    0.481    pre0_mod0_post0
##  6 Fold06 rsq     standard    0.202    pre0_mod0_post0
##  7 Fold07 rsq     standard    0.0182   pre0_mod0_post0
##  8 Fold08 rsq     standard    0.158    pre0_mod0_post0
##  9 Fold09 rsq     standard    0.00172  pre0_mod0_post0
## 10 Fold10 rsq     standard    0.0740   pre0_mod0_post0
```

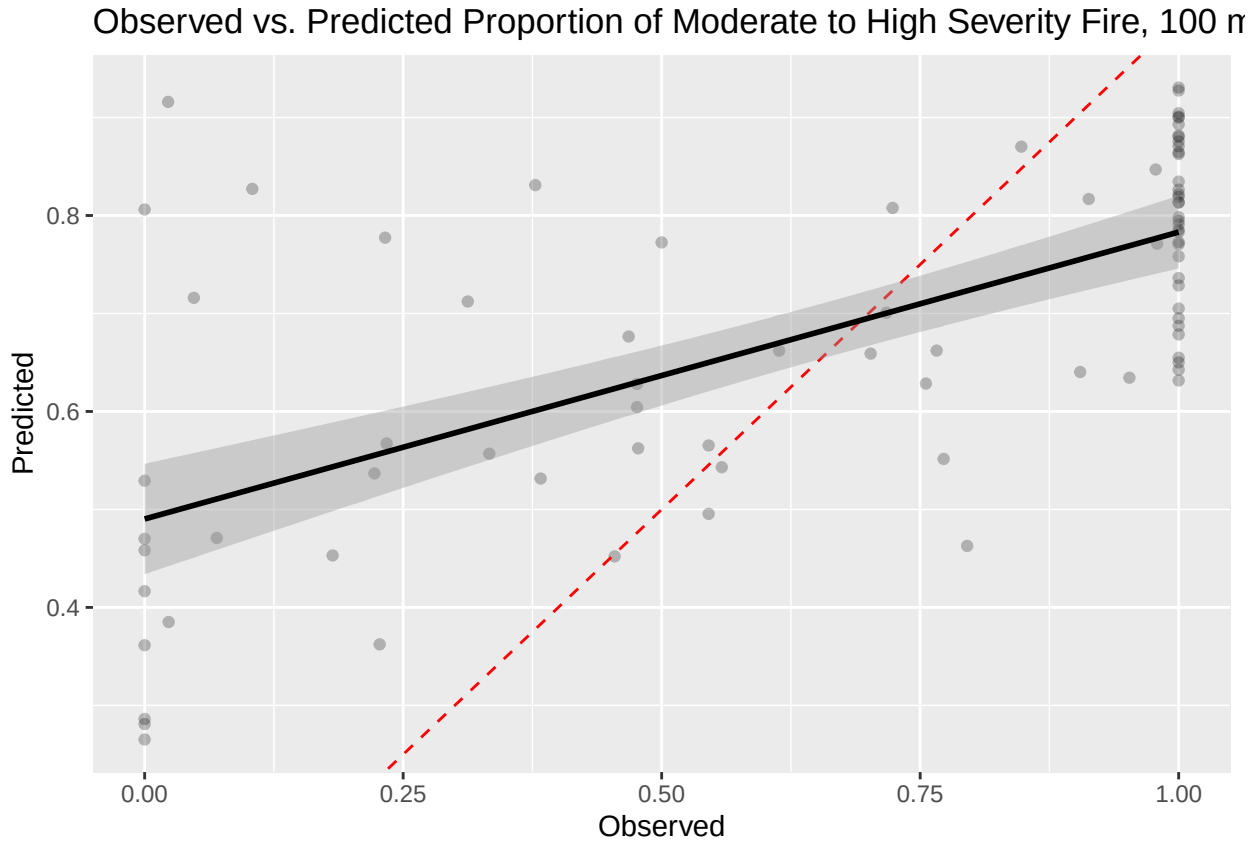
Fit Final Model

```
final_fit <- final_workflow %>%
  last_fit(data_split_mohi100)

collect_metrics(final_fit)      # test-set RMSE, R², etc.
```

```
## # A tibble: 2 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>          <dbl> <chr>
## 1 rmse    standard       0.299 pre0_mod0_post0
## 2 rsq     standard       0.441 pre0_mod0_post0
```

```r
mohi100_predictions <- collect_predictions(final_fit)  # observed vs. predicted on the held-out test se
ggplot(mohi100_predictions, aes(prop_mod_high_sev, .pred)) +
  geom_point(alpha = 0.25)+
  geom_abline(linetype = "dashed", color = "red") +
  geom_smooth(method = "lm", color = "black") +
  labs(
    x = "Observed",
    y = "Predicted",
    color = NULL,
    title = "Observed vs. Predicted Proportion of Moderate to High Severity Fire, 100 m"
  )
```




```r
final_fitted <- final_fit$.workflow[[1]]
final_fitted
```

```
## == Workflow [trained] ==========================================================
## Preprocessor: Recipe
## Model: rand_forest()
##
## -- Preprocessor ----------------------------------------------------------------
## 0 Recipe Steps
##
## -- Model -----------------------------------------------------------------------
## Ranger result
##
```

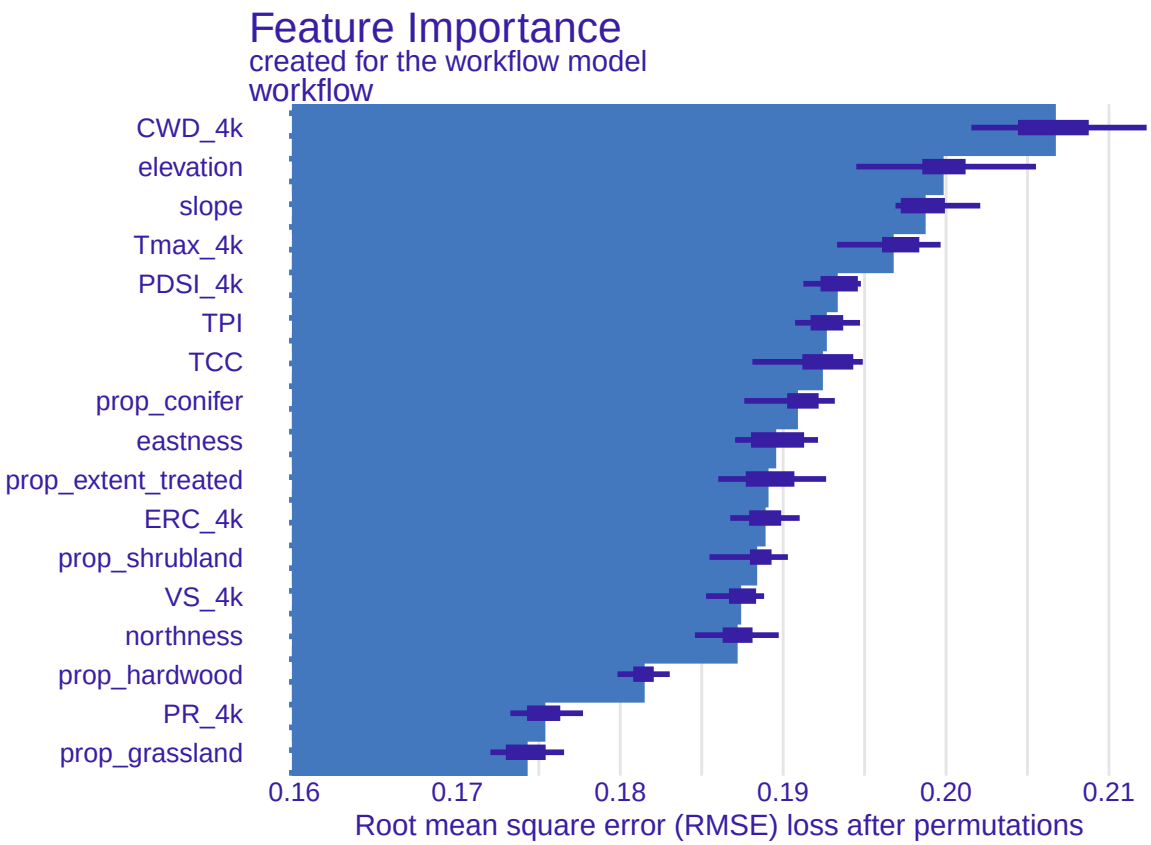
```
## Call:
##  ranger::ranger(x = maybe_data_frame(x), y = y, mtry = min_cols(~1L,      x), num.trees = ~2000, min
##
## Type:                             Regression
## Number of trees:                  2000
## Sample size:                      324
## Number of independent variables:  17
## Mtry:                             1
## Target node size:                 1
## Variable importance mode:         permutation
## Splitrule:                        variance
## OOB prediction error (MSE):       0.0978673
## R squared (OOB):                  0.3320594
```

Explainer

```
mohi100_explainer <- explain_tidymodels(
  final_fitted,
  data = mohi100_train %>% st_drop_geometry() %>% dplyr::select(all_of(preds), Incid_Name),
  y = mohi100_train$prop_mod_high_sev,
  verbose = FALSE
)
```

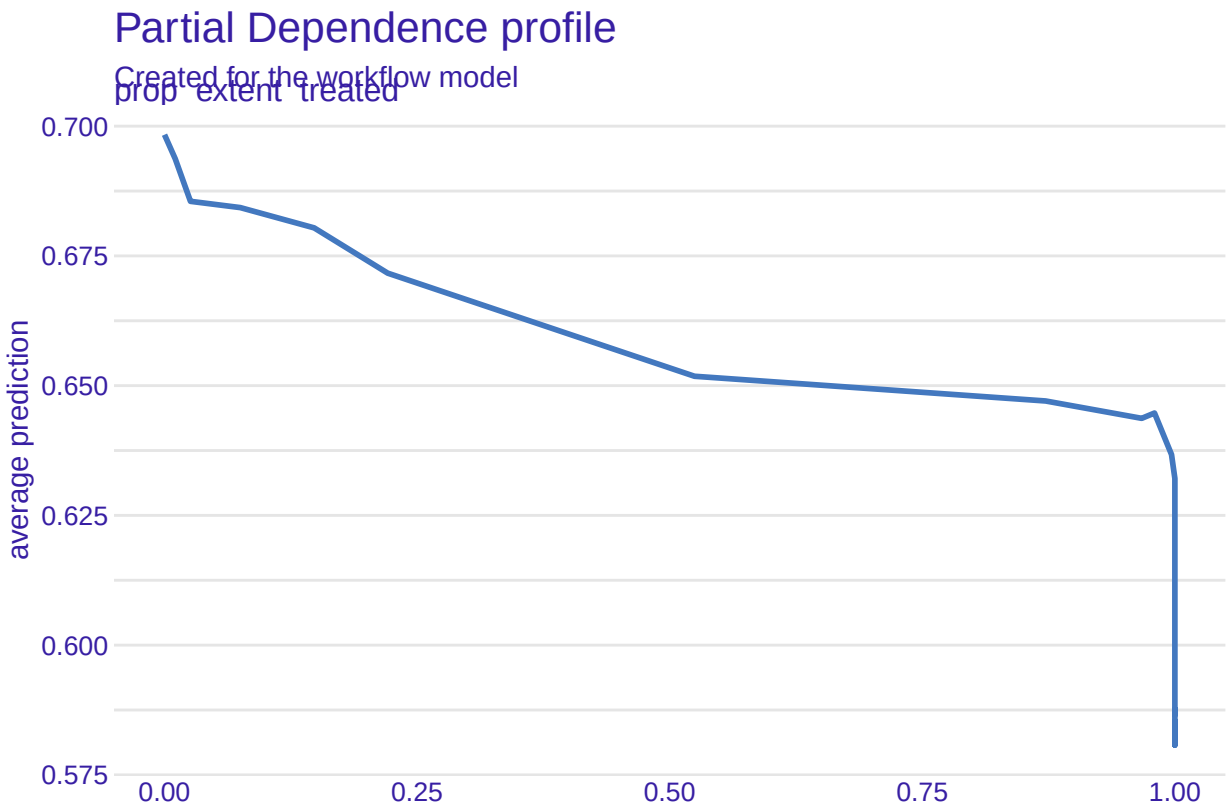
Variable Importance Plot

```
# Variable Importance Plots
vip_mohi100 <- variable_importance(mohi100_explainer,
                                   variables = preds)
plot(vip_mohi100)
```



PDP

```
pdp_mohi100 <- model_profile(
  mohi100_explainer,
  variables = "prop_extent_treated",
  N = NULL
)
plot(pdp_mohi100)
```



250m

```
set.seed(123)
data_split_mohi250 <- initial_split(plots_250m, strata = "prop_mod_high_sev", prop = 0.8)
mohi250_train <- training(data_split_mohi250)
mohi250_test  <- testing(data_split_mohi250)
```

Create a recipe

```
mohi250_rec <- recipe(prop_mod_high_sev ~ northness + eastness + elevation + TPI + slope + Tmax_4k + PR

mohi250_prep <- prep(mohi250_rec)
```

Set up a model specification to tune

```
tune_spec <- rand_forest(mtry = tune(),
                         trees = 2000,
                         min_n = tune()) %>%
             set_mode("regression") %>%
             set_engine("ranger", importance = "permutation")
```

Set up a workflow

```
tune_wf <- workflow() %>%
  add_recipe(mohi250_rec) %>%
  add_model(tune_spec)
```

Tune hyperparameters based on RMSE

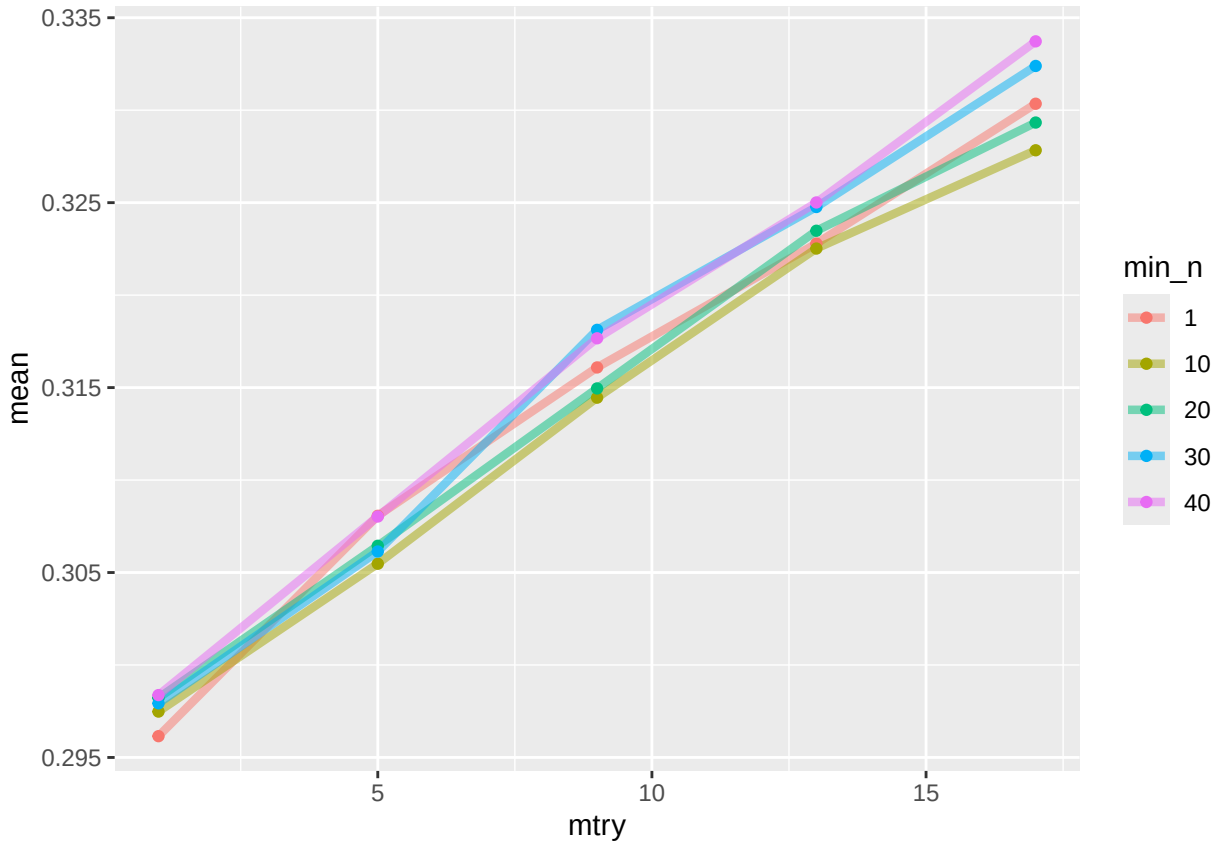
```
set.seed(234)
mohi250_folds <- spatial_clustering_cv(mohi250_train)

doParallel::registerDoParallel()
#show_notes(.Last.tune.result)

rf_grid <- grid_regular(
  mtry(range = c(1, 17)),
  min_n(range = c(1,40)),
  levels = 5
)

set.seed(456)
regular_res <- tune_grid(
  tune_wf,
  resamples = mohi250_folds,
  grid = rf_grid
)

regular_res %>%
  collect_metrics() %>%
  filter(.metric == "rmse") %>%
  mutate(min_n = factor(min_n)) %>%
  ggplot(aes(mtry, mean, color = min_n)) +
  geom_line(alpha = 0.5, size = 1.5) +
  geom_point()
```



Select Best Fit

```
best_rmse <- select_best(regular_res, metric = "rmse")
final_workflow <- tune_wf %>%
  finalize_workflow(best_rmse)
```

Evaluate metrics across spatial crossvalidation folds

```
rf_res <-
    final_workflow %>%
    fit_resamples(
      resamples = mohi250_folds,
      metrics = metric_set(rmse, rsq),
      control = control_resamples(save_pred = TRUE)
      )

rf_res %>%
  collect_metrics(summarize = TRUE) %>%
  mutate(Dataset="training") %>%
  mutate(Model="RF")
```

```
## # A tibble: 2 x 8
##   .metric .estimator  mean     n std_err .config         Dataset  Model
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>           <chr>    <chr>
## 1 rmse    standard   0.296    10  0.0268 pre0_mod0_post0 training RF
## 2 rsq     standard   0.120    10  0.0264 pre0_mod0_post0 training RF
```

```
rf_res %>%
  collect_metrics(summarize = FALSE) %>%
  filter(.metric == "rmse")
```

```
## # A tibble: 10 x 5
##    id     .metric .estimator .estimate .config
##    <chr>  <chr>   <chr>          <dbl> <chr>
##  1 Fold01 rmse    standard       0.225 pre0_mod0_post0
##  2 Fold02 rmse    standard       0.303 pre0_mod0_post0
##  3 Fold03 rmse    standard       0.262 pre0_mod0_post0
##  4 Fold04 rmse    standard       0.322 pre0_mod0_post0
##  5 Fold05 rmse    standard       0.280 pre0_mod0_post0
##  6 Fold06 rmse    standard       0.204 pre0_mod0_post0
##  7 Fold07 rmse    standard       0.234 pre0_mod0_post0
##  8 Fold08 rmse    standard       0.448 pre0_mod0_post0
##  9 Fold09 rmse    standard       0.436 pre0_mod0_post0
## 10 Fold10 rmse    standard       0.248 pre0_mod0_post0
```

```
rf_res %>%
  collect_metrics(summarize = FALSE) %>%
  dplyr::filter(.metric == "rsq")
```

```
## # A tibble: 10 x 5
##    id     .metric .estimator .estimate .config
##    <chr>  <chr>   <chr>          <dbl> <chr>
##  1 Fold01 rsq     standard      0.142  pre0_mod0_post0
##  2 Fold02 rsq     standard      0.184  pre0_mod0_post0
##  3 Fold03 rsq     standard      0.176  pre0_mod0_post0
##  4 Fold04 rsq     standard      0.0376 pre0_mod0_post0
##  5 Fold05 rsq     standard      0.0523 pre0_mod0_post0
##  6 Fold06 rsq     standard      0.208  pre0_mod0_post0
##  7 Fold07 rsq     standard      0.0962 pre0_mod0_post0
##  8 Fold08 rsq     standard      0.250  pre0_mod0_post0
##  9 Fold09 rsq     standard      0.0265 pre0_mod0_post0
## 10 Fold10 rsq     standard      0.0229 pre0_mod0_post0
```

Fit Final Model

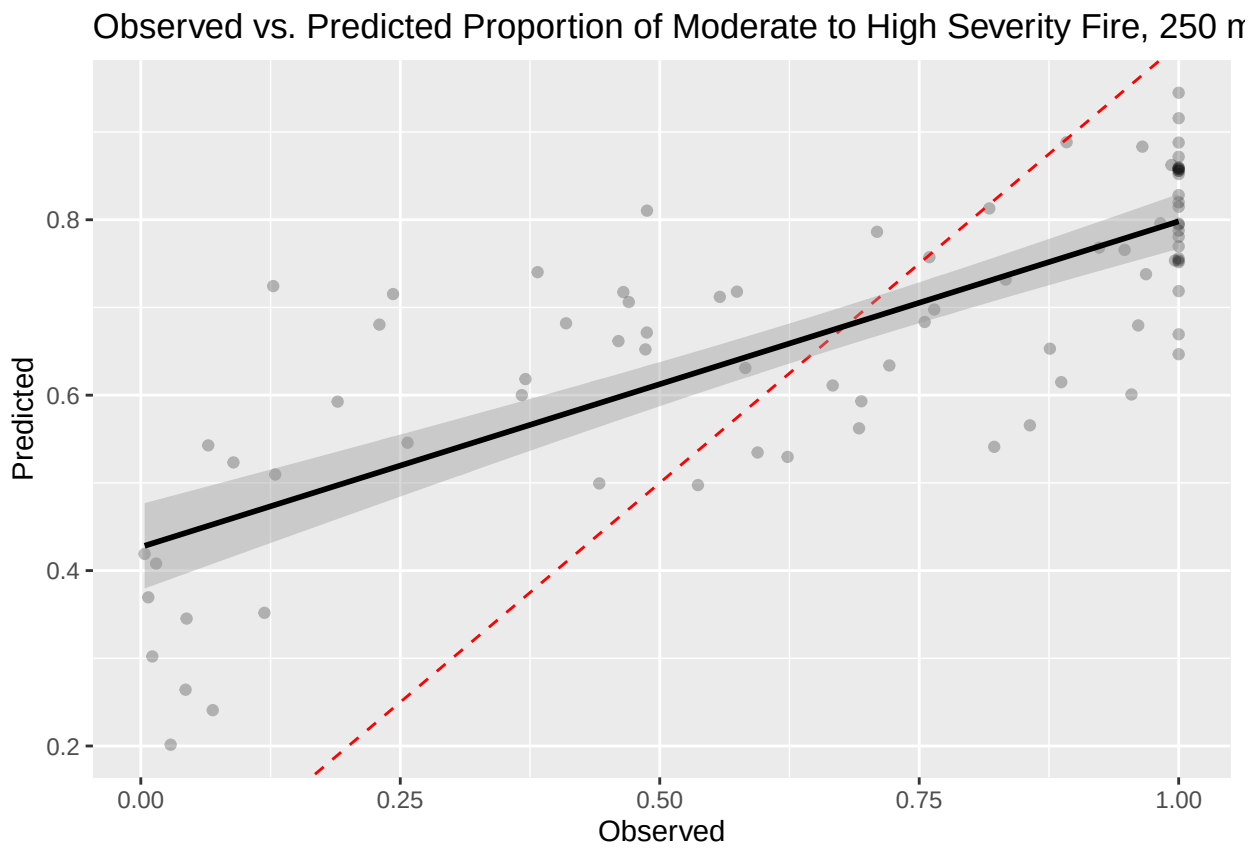
```
final_fit <- final_workflow %>%
  last_fit(data_split_mohi250)

collect_metrics(final_fit)      # test-set RMSE, R², etc.
```

```
## # A tibble: 2 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>          <dbl> <chr>
## 1 rmse    standard       0.242 pre0_mod0_post0
## 2 rsq     standard       0.615 pre0_mod0_post0
```

```
mohi250_predictions <- collect_predictions(final_fit)  # observed vs. predicted on the held-out test se
ggplot(mohi250_predictions, aes(prop_mod_high_sev, .pred)) +
```

```
  geom_point(alpha = 0.25)+
  geom_abline(linetype = "dashed", color = "red") +
  geom_smooth(method = "lm", color = "black") +
  labs(
    x = "Observed",
    y = "Predicted",
    color = NULL,
    title = "Observed vs. Predicted Proportion of Moderate to High Severity Fire, 250 m"
  )
```




```
final_fitted <- final_fit$.workflow[[1]]
final_fitted
```

```
## == Workflow [trained] ==========================================================
## Preprocessor: Recipe
## Model: rand_forest()
##
## -- Preprocessor ----------------------------------------------------------------
## 0 Recipe Steps
##
## -- Model -----------------------------------------------------------------------
## Ranger result
##
## Call:
##  ranger::ranger(x = maybe_data_frame(x), y = y, mtry = min_cols(~1L,      x), num.trees = ~2000, min
```

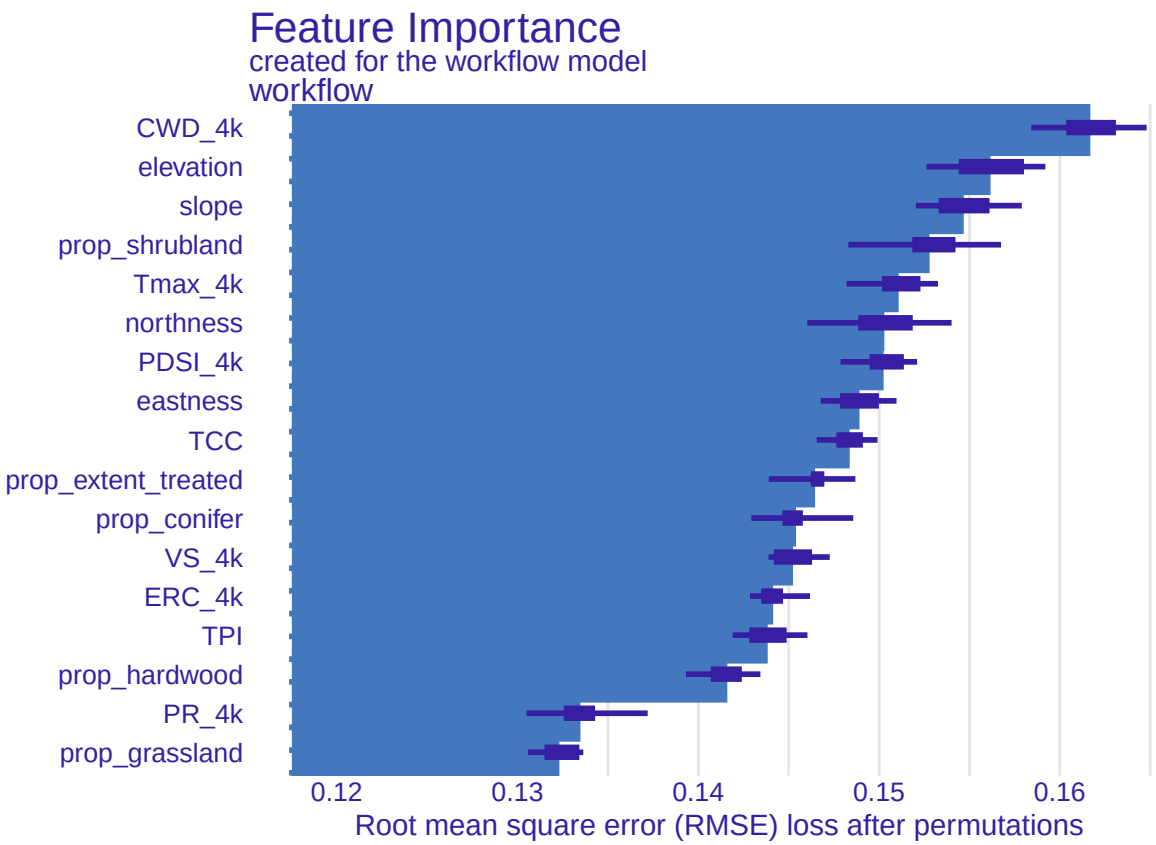
```
##
## Type:                             Regression
## Number of trees:                  2000
## Sample size:                      323
## Number of independent variables:  17
## Mtry:                             1
## Target node size:                 1
## Variable importance mode:         permutation
## Splitrule:                        variance
## OOB prediction error (MSE):       0.06751948
## R squared (OOB):                  0.3993978
```

Explainer

```
mohi250_explainer <- explain_tidymodels(
  final_fitted,
  data = mohi250_train %>% st_drop_geometry() %>% dplyr::select(all_of(preds), Incid_Name),
  y = mohi250_train$prop_mod_high_sev,
  verbose = FALSE
)
```

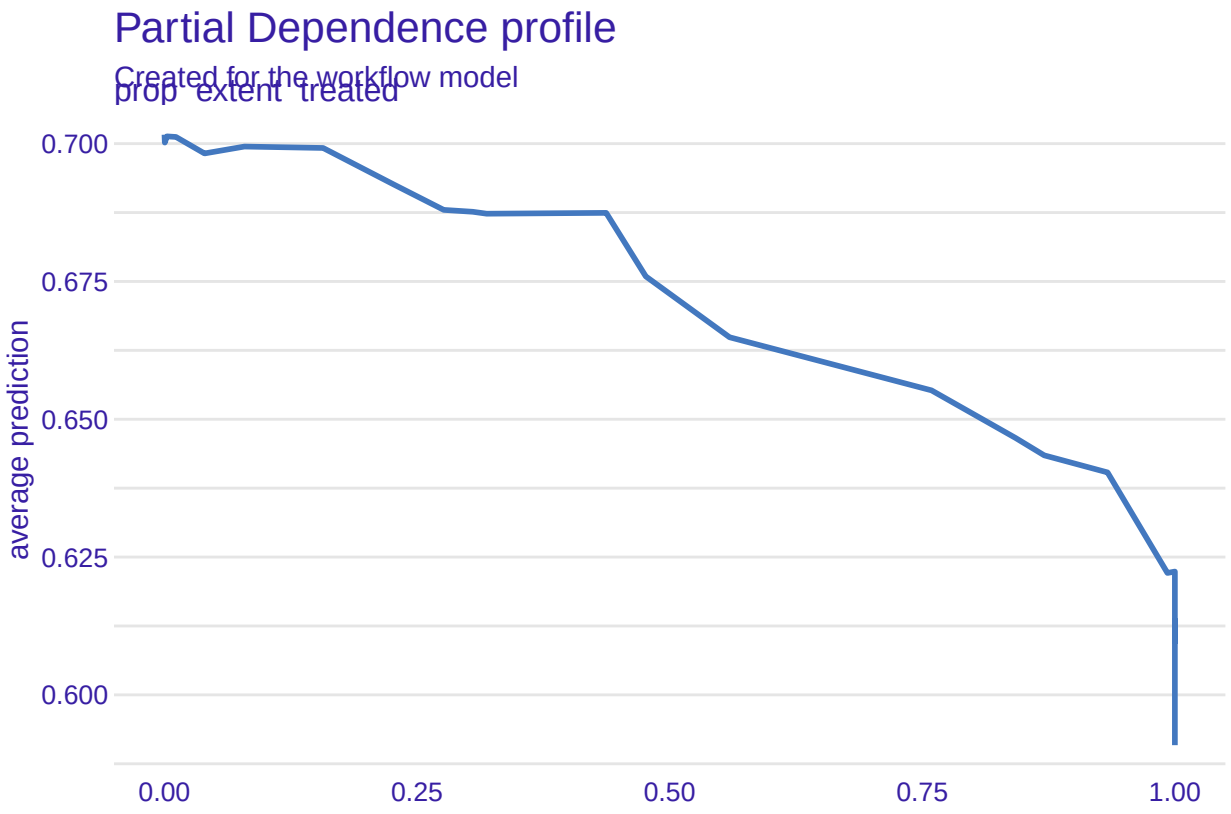
Variable Importance Plot

```
# Variable Importance Plots
vip_mohi250 <- variable_importance(mohi250_explainer,
                                   variables = preds)
plot(vip_mohi250)
```



PDP

```
pdp_mohi250 <- model_profile(
  mohi250_explainer,
  variables = "prop_extent_treated",
  N = NULL
)
plot(pdp_mohi250)
```



500m

```
set.seed(123)
data_split_mohi500 <- initial_split(plots_500m, strata = "prop_mod_high_sev", prop = 0.8)
mohi500_train <- training(data_split_mohi500)
mohi500_test  <- testing(data_split_mohi500)
```

Create a recipe

```
mohi500_rec <- recipe(prop_mod_high_sev ~ northness + eastness + elevation + TPI + slope + Tmax_4k + PR

mohi500_prep <- prep(mohi500_rec)
```

Set up a model specification to tune

```
tune_spec <- rand_forest(mtry = tune(),
                         trees = 2000,
                         min_n = tune()) %>%
             set_mode("regression") %>%
             set_engine("ranger", importance = "permutation")
```

Set up a workflow

```
tune_wf <- workflow() %>%
  add_recipe(mohi500_rec) %>%
  add_model(tune_spec)
```

Tune hyperparameters based on RMSE

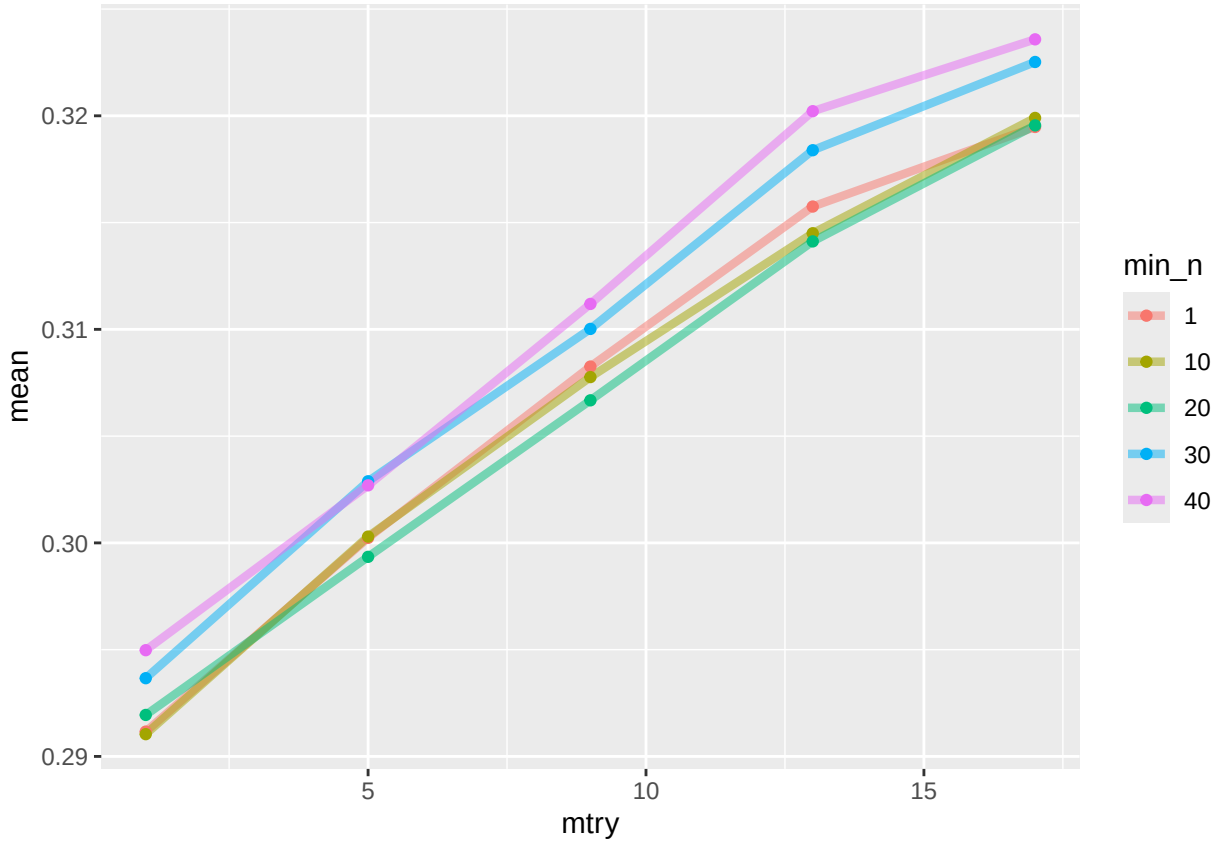
```
set.seed(234)
mohi500_folds <- spatial_clustering_cv(mohi500_train)

doParallel::registerDoParallel()
#show_notes(.Last.tune.result)

rf_grid <- grid_regular(
  mtry(range = c(1, 17)),
  min_n(range = c(1,40)),
  levels = 5
)

set.seed(456)
regular_res <- tune_grid(
  tune_wf,
  resamples = mohi500_folds,
  grid = rf_grid
)

regular_res %>%
  collect_metrics() %>%
  filter(.metric == "rmse") %>%
  mutate(min_n = factor(min_n)) %>%
  ggplot(aes(mtry, mean, color = min_n)) +
  geom_line(alpha = 0.5, size = 1.5) +
  geom_point()
```



Select Best Fit

```
best_rmse <- select_best(regular_res, metric = "rmse")
final_workflow <- tune_wf %>%
  finalize_workflow(best_rmse)
```

Evaluate metrics across spatial crossvalidation folds

```
rf_res <-
    final_workflow %>%
    fit_resamples(
      resamples = mohi500_folds,
      metrics = metric_set(rmse, rsq),
      control = control_resamples(save_pred = TRUE)
      )

rf_res %>%
  collect_metrics(summarize = TRUE) %>%
  mutate(Dataset="training") %>%
  mutate(Model="RF")
```

```
## # A tibble: 2 x 8
##   .metric .estimator  mean     n std_err .config         Dataset  Model
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>           <chr>    <chr>
## 1 rmse    standard   0.291    10  0.0391 pre0_mod0_post0 training RF
## 2 rsq     standard   0.199    10  0.0398 pre0_mod0_post0 training RF
```

```
rf_res %>%
  collect_metrics(summarize = FALSE) %>%
  filter(.metric == "rmse")
```

```
## # A tibble: 10 x 5
##    id     .metric .estimator .estimate .config
##    <chr>  <chr>   <chr>          <dbl> <chr>
##  1 Fold01 rmse    standard       0.280 pre0_mod0_post0
##  2 Fold02 rmse    standard       0.405 pre0_mod0_post0
##  3 Fold03 rmse    standard       0.190 pre0_mod0_post0
##  4 Fold04 rmse    standard       0.262 pre0_mod0_post0
##  5 Fold05 rmse    standard       0.196 pre0_mod0_post0
##  6 Fold06 rmse    standard       0.195 pre0_mod0_post0
##  7 Fold07 rmse    standard       0.215 pre0_mod0_post0
##  8 Fold08 rmse    standard       0.364 pre0_mod0_post0
##  9 Fold09 rmse    standard       0.574 pre0_mod0_post0
## 10 Fold10 rmse    standard       0.233 pre0_mod0_post0
```

```
rf_res %>%
  collect_metrics(summarize = FALSE) %>%
  dplyr::filter(.metric == "rsq")
```

```
## # A tibble: 10 x 5
##    id     .metric .estimator .estimate .config
##    <chr>  <chr>   <chr>          <dbl> <chr>
##  1 Fold01 rsq     standard      0.260  pre0_mod0_post0
##  2 Fold02 rsq     standard      0.208  pre0_mod0_post0
##  3 Fold03 rsq     standard      0.0552 pre0_mod0_post0
##  4 Fold04 rsq     standard      0.0620 pre0_mod0_post0
##  5 Fold05 rsq     standard      0.431  pre0_mod0_post0
##  6 Fold06 rsq     standard      0.139  pre0_mod0_post0
##  7 Fold07 rsq     standard      0.149  pre0_mod0_post0
##  8 Fold08 rsq     standard      0.296  pre0_mod0_post0
##  9 Fold09 rsq     standard      0.0722 pre0_mod0_post0
## 10 Fold10 rsq     standard      0.316  pre0_mod0_post0
```

Fit Final Model

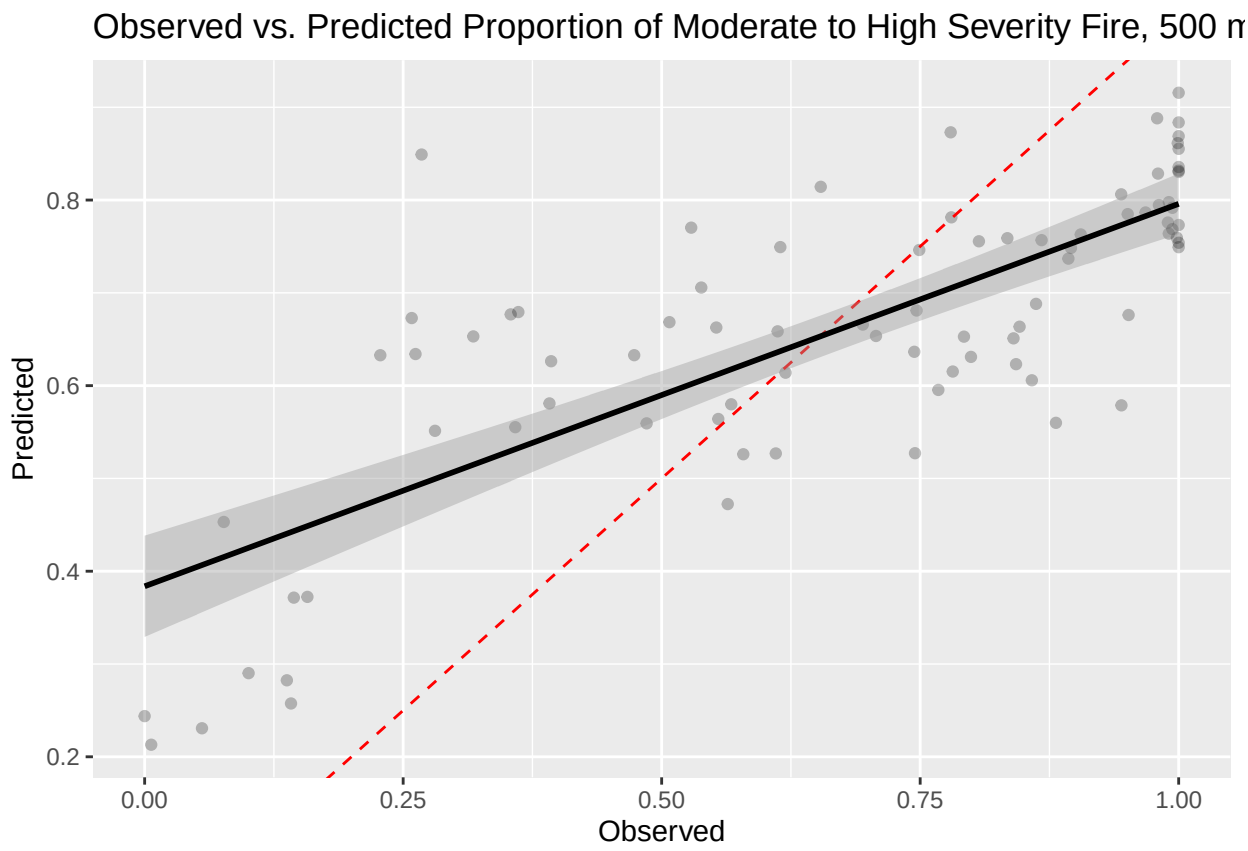
```
final_fit <- final_workflow %>%
  last_fit(data_split_mohi500)

collect_metrics(final_fit)      # test-set RMSE, R², etc.
```

```
## # A tibble: 2 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>          <dbl> <chr>
## 1 rmse    standard       0.206 pre0_mod0_post0
## 2 rsq     standard       0.602 pre0_mod0_post0
```

```
mohi500_predictions <- collect_predictions(final_fit)  # observed vs. predicted on the held-out test se
ggplot(mohi500_predictions, aes(prop_mod_high_sev, .pred)) +
```

```
  geom_point(alpha = 0.25)+
  geom_abline(linetype = "dashed", color = "red") +
  geom_smooth(method = "lm", color = "black") +
  labs(
    x = "Observed",
    y = "Predicted",
    color = NULL,
    title = "Observed vs. Predicted Proportion of Moderate to High Severity Fire, 500 m"
  )
```




```
final_fitted <- final_fit$.workflow[[1]]
final_fitted
```

```
## == Workflow [trained] ==========================================================
## Preprocessor: Recipe
## Model: rand_forest()
##
## -- Preprocessor ----------------------------------------------------------------
## 0 Recipe Steps
##
## -- Model -----------------------------------------------------------------------
## Ranger result
##
## Call:
##  ranger::ranger(x = maybe_data_frame(x), y = y, mtry = min_cols(~1L,      x), num.trees = ~2000, min
```

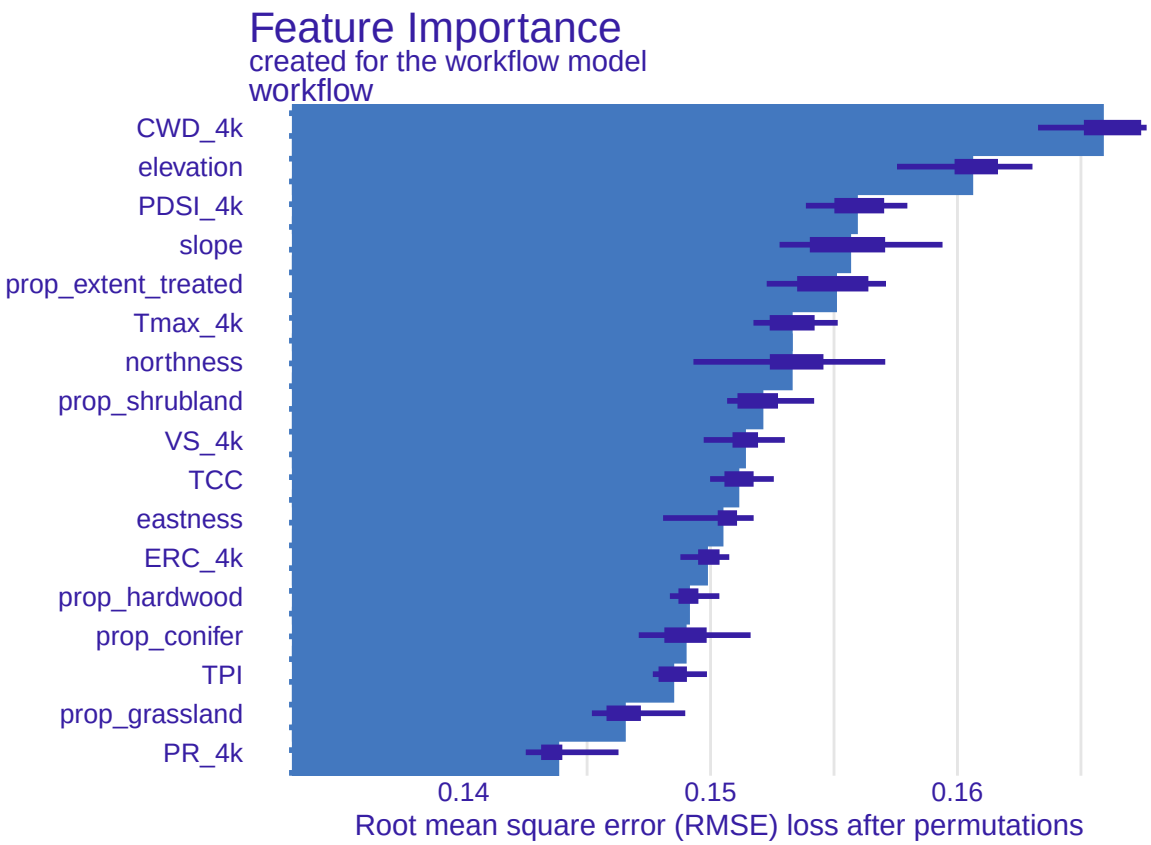
```
##
## Type:                             Regression
## Number of trees:                  2000
## Sample size:                      322
## Number of independent variables:  17
## Mtry:                             1
## Target node size:                 10
## Variable importance mode:         permutation
## Splitrule:                        variance
## OOB prediction error (MSE):       0.04592041
## R squared (OOB):                  0.5237789
```

Explainer

```
mohi500_explainer <- explain_tidymodels(
  final_fitted,
  data = mohi500_train %>% st_drop_geometry() %>% dplyr::select(all_of(preds), Incid_Name),
  y = mohi500_train$prop_mod_high_sev,
  verbose = FALSE
)
```

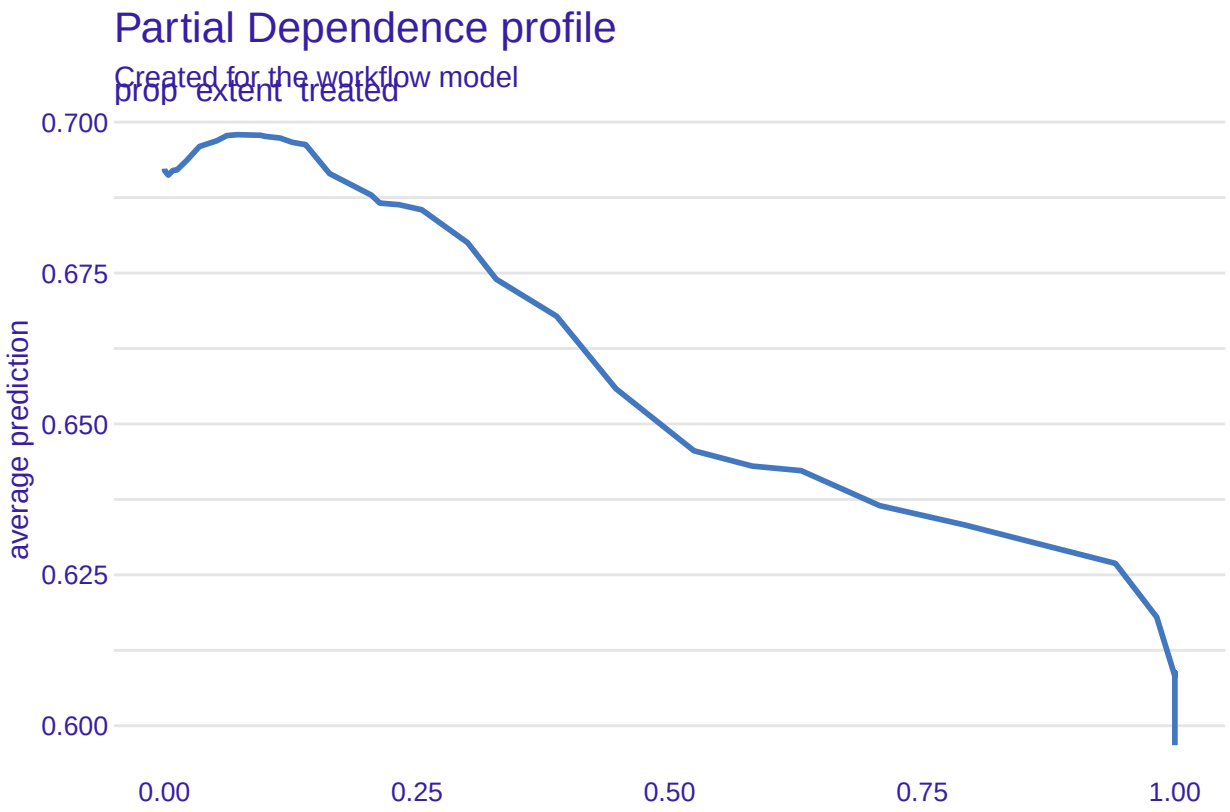
Variable Importance Plot

```
# Variable Importance Plots
vip_mohi500 <- variable_importance(mohi500_explainer,
                                   variables = preds)
plot(vip_mohi500)
```



PDP

```
pdp_mohi500 <- model_profile(
  mohi500_explainer,
  variables = "prop_extent_treated",
  N = NULL
)
plot(pdp_mohi500)
```



Figures

Variable Importance, Proportion of High Severity

Partial Dependence Plot, Proportion of High Severity

Variable Importance, Proportion of Moderate-High Severity

Partial Dependence Plot, Proportion of Moderate-High Severity

Testing out interaction pdp

Average reduction in percent area burned at high severity between 0 and 1 treated area

```
trt_effect <- function(explainer) {
  dat <- explainer$data

  d0 <- dat; d0$prop_extent_treated <- 0   # untreated
  d1 <- dat; d1$prop_extent_treated <- 1   # fully treated
```

```
  p0 <- predict(explainer$model, new_data = d0)$.pred
  p1 <- predict(explainer$model, new_data = d1)$.pred

  tibble::tibble(
    mean_pred_untreated = mean(p0),
    mean_pred_treated   = mean(p1),
    abs_diff            = mean(p1) - mean(p0),                # in proportion units
    pct_diff            = (mean(p1) - mean(p0)) / mean(p0) * 100  # relative % change
  )
}

results <- dplyr::bind_rows(
  "High sev, 100m"   = trt_effect(hi100_explainer),
  "High sev, 250m"   = trt_effect(hi250_explainer),
  "High sev, 500m"   = trt_effect(hi500_explainer),
  "Mod-high, 100m"   = trt_effect(mohi100_explainer),
  "Mod-high, 250m"   = trt_effect(mohi250_explainer),
  "Mod-high, 500m"   = trt_effect(mohi500_explainer),
  .id = "model"
)
results
```

```
## # A tibble: 6 x 5
##   model          mean_pred_untreated mean_pred_treated abs_diff pct_diff
##   <chr>                        <dbl>             <dbl>    <dbl>    <dbl>
## 1 High sev, 100m               0.471             0.354  -0.117     -24.8
## 2 High sev, 250m               0.467             0.377  -0.0898    -19.2
## 3 High sev, 500m               0.466             0.404  -0.0613    -13.2
## 4 Mod-high, 100m               0.698             0.583  -0.115     -16.5
## 5 Mod-high, 250m               0.702             0.610  -0.0913    -13.0
## 6 Mod-high, 500m               0.692             0.604  -0.0888    -12.8
```

```
mean(results$pct_diff)
```

```
## [1] -16.5964
```